

Certification professionnelle

Titre professionnel: **Concepteur développeur d'applications**

RNCP: **37873**

DOSSIER PROJET

PROJET :

Tempo

Rédacteur :

Valentin Musset

Date : *7 Octobre 2026*

SOMMAIRE

1. LISTE DES COMPÉTENCES DU RÉFÉRENTIEL COUVERTES PAR LE PROJET.....	7
1.1 Développer une application sécurisée.....	7
1.1.1. Installer et configurer son environnement de travail en fonction du projet.....	7
1.1.2. Développer des interfaces utilisateur.....	7
1.1.3. Développer des composants métier.....	7
1.1.4. Contribuer à la gestion d'un projet informatique.....	8
1.2. Concevoir et développer une application sécurisée organisée en couches.....	8
1.2.1. Analyser les besoins et maquetter une application.....	8
1.2.2. Définir l'architecture logicielle d'une application.....	8
1.2.3. Concevoir et mettre en place une base de données relationnelle.....	9
1.2.4. Développer des composants d'accès aux données SQL et NoSQL.....	9
1.3. Préparer le déploiement d'une application sécurisée.....	9
1.3.1. Préparer et exécuter les plans de tests d'une application.....	9
1.3.2. Préparer et documenter le déploiement d'une application.....	10
1.3.3. Contribuer à la mise en production dans une démarche DevOps.....	10
2. CAHIER DES CHARGES.....	11
2.1. Description de l'existant.....	11
2.2. Reprise de l'existant.....	11
2.3. Principes de référencement.....	11
2.4. Exigences de performances et de volumétrie.....	11
2.5. Multilinguisme et adaptations pour un public spécifique.....	11
2.6. Description graphique et ergonomique.....	11
2.6.1. Composants de la charte graphique.....	11
2.6.2. Responsive design et ergonomie.....	12
2.7. Besoins fonctionnels métier.....	13

2.7.1. Utilisateurs du projet.....	13
2.7.2. Informations relatives aux contenus.....	14
2.7.3. Besoins fonctionnels.....	14
2.8. Budget.....	14
3. PRÉSENTATION DE L'ENTREPRISE ET DU SERVICE.....	15
3.1. Présentation de l'entreprise et du service.....	15
3.2. Objectifs du projet.....	15
3.3. Cible adressée par le projet.....	15
3.4. Processus utilisateur impacté.....	15
4. GESTION DE PROJET.....	16
4.1. Intervenants sur le projet.....	16
4.2. Méthodologie.....	16
4.3. Outils, planning et suivi.....	16
4.4. Objectifs de qualité.....	17
5. SPÉCIFICATIONS FONCTIONNELLES.....	19
5.1. Contraintes du projet et livrables attendus.....	19
5.1.1. Criticité de l'application.....	19
5.1.2. Applications connexes.....	19
5.1.3. Services tiers.....	19
5.1.4. Livrables attendus.....	19
5.2. Architecture logicielle du projet.....	19
5.3. Maquettes et enchaînement des écrans.....	20
5.3.1. Cartographie.....	20
5.3.2. Maquettes.....	20
5.4. Modélisation des données.....	22
5.4.1. MCD (MERISE).....	22
5.4.2. MLD (MERISE).....	22

5.4.3. MPD (MERISE).....	23
5.4.4. Diagramme de classes (UML).....	23
5.5. Création et modification de la base de données.....	24
5.5.1. Script de création.....	24
5.5.2. Choix retenus.....	24
5.5.3. Scripts de modification.....	25
5.5.4. Justification des contraintes.....	25
5.6. Diagrammes de comportement.....	26
5.6.1. Diagramme de cas d'utilisation global (UML).....	26
5.7. Fonctionnalités détaillées les plus significatives.....	27
5.7.1. Fonctionnalité 1 : réserver un espace.....	27
5.7.2. Fonctionnalité 2 : annuler une réservation.....	30
5.7.3. Fonctionnalité 3 : gérer les espaces.....	32
5.7.4. Fonctionnalité 4 : effectuer un check-in par QR code.....	35
5.7.5. Fonctionnalité 5 : s'authentifier.....	37
5.7.6. Fonctionnalité 6 : consulter les statistiques d'occupation.....	38
5.7.7. Fonctionnalité 7 : consulter les audits.....	38
5.7.8. Fonctionnalité 8 : inviter et gérer les participants.....	38
6. SPÉCIFICATIONS TECHNIQUES.....	39
6.1. Référencement.....	39
6.2. Environnement technique.....	39
6.2.1. Ports et configuration.....	40
6.2.2. Déploiement public du 22 septembre 2026.....	40
6.3. Navigation et accessibilité.....	42
6.3.1. Routes de l'API.....	42
6.4. Services tiers.....	43
6.5. Sécurité.....	43

6.5.1. Limites connues.....	44
7. RÉALISATIONS.....	45
7.1. Détection des chevauchements.....	45
7.1.1. Affichage.....	45
7.1.2. Extrait de code.....	45
7.1.3. Argumentation.....	46
7.2. Authentification et contrôle des rôles.....	46
7.2.1. Affichage.....	46
/.....	46
7.2.2. Extrait de code.....	46
7.2.3. Argumentation.....	46
7.3. Capacité et concurrence des participants.....	47
7.3.1. Affichage.....	47
7.3.2. Extrait de code.....	47
7.3.3. Argumentation.....	47
7.4. Jeton de check-in et stockage du hash.....	48
7.4.1. Affichage.....	48
7.4.2. Extrait de code.....	49
7.4.3. Argumentation.....	49
7.5. Audits MongoDB.....	49
7.5.1. Affichage.....	49
7.5.2. Extrait de code.....	49
7.5.3. Argumentation.....	50
7.6. Client RPC typé.....	50
7.6.1. Affichage.....	50
/.....	50
7.6.2. Extrait de code.....	50

7.6.3. Argumentation.....	50
8. ÉLÉMENTS DE SÉCURITÉ DE L'APPLICATION.....	51
9. PLAN DE TESTS.....	52
9.1. Niveaux de test.....	52
9.2. Couverture par module.....	52
9.3. Environnements et critères de réussite.....	53
9.3.1. Vérifications du déploiement du 22 septembre 2026.....	53
9.4. Évolutions du plan.....	54
10. JEU D'ESSAI DE LA FONCTIONNALITÉ LA PLUS REPRÉSENTATIVE.....	55
10.1. Fonctionnalité retenue.....	55
10.2. Scénarios.....	55
10.3. Résultats.....	56
10.4. Conclusion.....	57
11. VEILLE SUR LES VULNÉRABILITÉS DE SÉCURITÉ.....	58

1. LISTE DES COMPÉTENCES DU RÉFÉRENTIEL COUVERTES PAR LE PROJET

Dans ce dossier, j'explique comment j'ai conçu, développé et préparé le déploiement de Tempo, jusqu'à sa mise en ligne le 22 septembre 2026 sur <https://tempo-val.duckdns.org>. Je m'appuie sur le code du dépôt, les tests et les écrans de l'application pour décrire mon travail.

1.1 Développer une application sécurisée

1.1.1. Installer et configurer son environnement de travail en fonction du projet

J'ai regroupé le backend Hono et le frontend SvelteKit dans un même dépôt Bun. Depuis la racine, je peux lancer le développement, les tests, le lint, le formatage et le build pour les deux applications. Chacune garde aussi ses propres commandes si je veux travailler dessus séparément.

Je peux lancer PostgreSQL et MongoDB en local ou avec Docker Compose. Les versions de Bun et des images de base sont fixées pour réduire les différences entre mon poste et GitHub Actions. Les identifiants des bases et le secret JWT se trouvent dans des fichiers d'environnement ignorés par Git. Les fichiers .env.example donnent les variables à renseigner, avec des valeurs factices.

J'utilise Git pour conserver l'historique du code, Oxlint pour le lint et Oxfmt pour le formatage. Bun Test et Vitest exécutent les tests unitaires ; Playwright lance les parcours complets dans Chromium. J'ai regroupé dans README.md les commandes de démarrage, de migration, de seed, de sauvegarde et de restauration.

1.1.2. Développer des interfaces utilisateur

Pour construire les écrans, j'ai utilisé Svelte 5, SvelteKit, Tailwind CSS et shadcn-svelte. Les états des pages sont gérés avec les runes Svelte. Lorsqu'un formulaire envoie une demande, il indique le chargement en cours. Si l'API refuse la demande, par exemple parce que l'espace est déjà réservé ou qu'une invitation ne peut pas être créée, le message d'erreur s'affiche dans le formulaire.

L'interface s'adapte au rôle connecté. Un collaborateur ne voit ni les liens ni les actions d'administration. Un administrateur accède à la gestion des utilisateurs et des espaces, aux statistiques, aux audits et à la vue globale des réservations. La page des réservations permet aussi de choisir la visibilité, d'inviter un participant, de répondre à une invitation, de rejoindre une réservation publique et de générer un QR code.

J'ai regroupé les appels à l'API dans un client Hono RPC qui utilise le type AppType. Les réponses 401 et 403 y sont traitées au même endroit. Si l'utilisateur doit se connecter avant son check-in, le jeton QR est conservé dans le stockage de session pour le retour à cette page. Il n'est pas ajouté à la requête HTTP vers /login. Les tests Vitest et les contrôles Svelte vérifient ces comportements.

1.1.3. Développer des composants métier

J'ai organisé le backend en six modules : auth, users, workspaces, bookings, analytics et audit. Dans chacun, les routes HTTP sont séparées des traitements métier et de l'accès aux données. Une requête doit d'abord passer la validation Zod avant d'arriver dans un service.

Le module de réservation vérifie les créneaux et renvoie HTTP 409 lorsqu'une réservation entre en conflit avec une autre dans PostgreSQL. Il gère aussi la visibilité publique ou privée, les invitations et le nombre de participants. Avant d'ajouter quelqu'un, la transaction verrouille l'espace puis la réservation. Une modification de capacité ne peut donc pas se faire en même temps sans tenir compte de cet ajout. Le serveur crée un jeton aléatoire de 256 bits pour le check-in et garde uniquement son hash SHA-256. Pour confirmer sa présence, un utilisateur doit déjà être accepté dans la réservation et se trouver dans le créneau prévu. Si un nouveau QR est généré, l'ancien cesse de fonctionner. Je vérifie ces règles avec les tests de routes et un scénario d'intégration sur PostgreSQL.

1.1.4. Contribuer à la gestion d'un projet informatique

J'ai réalisé seul les différentes étapes du projet, depuis la définition du besoin jusqu'aux tests et à la préparation du déploiement, en passant par la conception et le développement. J'utilisais un tableau Kanban dans Trello pour découper ce travail en tâches.

J'ai commencé par la connexion, la création des espaces et la réservation. J'ai ensuite renforcé les contrôles d'accès et les règles en base de données, puis ajouté la CI et les tests d'intégration. Les invitations, le choix entre réservation publique ou privée et le check-in par QR code sont venus plus tard.

En local, je vérifie le formatage, le lint, les types, les tests et le build. GitHub Actions exécute aussi une recette avec Docker. Même si les contrôles passent sur mon poste, je dois encore vérifier le résultat de la CI pour la version que je remets.

1.2. Concevoir et développer une application sécurisée organisée en couches

1.2.1. Analyser les besoins et maquetter une application

J'ai d'abord décrit le besoin dans SPECS.md : ce qu'un collaborateur doit pouvoir réserver et ce qu'un administrateur doit pouvoir gérer, superviser ou consulter dans les audits. À partir de ces cas d'utilisation et des règles de gestion, j'ai préparé les modèles de données et les routes de l'API.

La première maquette Figma m'a servi à organiser la navigation. J'ai ajusté les écrans pendant le développement, selon les composants disponibles et les retours des tests. La section 5 reprend cette maquette, la liste des pages et les diagrammes de comportement.

1.2.2. Définir l'architecture logicielle d'une application

J'ai séparé l'application en trois couches : SvelteKit pour les écrans, Hono pour les routes et les traitements métier, PostgreSQL et MongoDB pour les données. Le frontend et le backend sont deux workspaces du même dépôt. Le client RPC utilise le type de l'application Hono : si une route ou les données attendues changent de façon incompatible, TypeScript le signale à la compilation.

Masquer une action dans le frontend ne suffit pas à en protéger l'accès. L'API vérifie les autorisations avec les middlewares chargés du JWT et des rôles. D'autres contrôles portent sur le CORS, les en-têtes de sécurité et le nombre de requêtes. Zod valide les données reçues, puis PostgreSQL applique les clés étrangères, les contraintes sur les dates et l'interdiction des réservations concurrentes.

J'ai gardé un seul backend, divisé en modules, pour limiter les services à maintenir. L'usage d'un runtime commun et d'images Docker multi-stage va dans le même sens. C'est sur ces choix de simplicité que repose ma démarche d'éco-conception pour Tempo. Je n'ai pas encore mesuré la tenue en charge de l'application.

1.2.3. Concevoir et mettre en place une base de données relationnelle

À partir du modèle de données, j'ai écrit le schéma Drizzle et généré les migrations SQL, conservées dans Git. PostgreSQL stocke les utilisateurs, les espaces, les réservations, les participants et les hashés des jetons QR. Les clés étrangères et les suppressions en cascade évitent de laisser des données sans parent. J'ai aussi ajouté les contraintes sur les dates, la capacité positive et les chevauchements de réservations d'un même espace.

Le seed crée deux comptes, plusieurs espaces et une réservation publique avec une invitation en attente. Je peux le relancer dans l'environnement de démonstration sans dupliquer ces données. Pour les tests unitaires, les appels aux bases sont remplacés par des mocks. Les tests d'intégration appliquent les migrations et utilisent une vraie base PostgreSQL.

Les volumes Docker conservent les données entre deux démarrages. Le README.md documente aussi les commandes `pg_dump` et `pg_restore` nécessaires à une sauvegarde ou à une restauration manuelle.

1.2.4. Développer des composants d'accès aux données SQL et NoSQL

Les services utilisent Drizzle pour exécuter les requêtes SQL avec les types du schéma. La création d'une réservation ou d'un participant se fait dans une transaction. Les requêtes chargent aussi les espaces, les propriétaires et les participants liés : le frontend reçoit ces relations sans devoir les reconstruire.

MongoDB conserve les audits de suppression : les données supprimées, l'auteur de l'action et sa date. Les événements s'affichent du plus récent au plus ancien. Si l'écriture dans MongoDB échoue, l'erreur est enregistrée dans les logs, mais la suppression déjà faite dans PostgreSQL est conservée. C'est la limite de cet audit dit best effort.

Quand un service renvoie une erreur métier, la route la traduit en statut HTTP. Les tests unitaires remplacent les dépendances des services par des mocks. Avec les tests d'intégration, je vérifie que les données sont bien enregistrées, que deux demandes concurrentes sont traitées correctement et que le parcours d'invitation jusqu'au check-in fonctionne. Ces tests contrôlent aussi l'ordre et le filtrage du journal d'audit.

1.3. Préparer le déploiement d'une application sécurisée

1.3.1. Préparer et exécuter les plans de tests d'une application

Les 106 tests unitaires et HTTP du backend et les 22 tests du frontend passent. Ils vérifient les services et les routes, mais aussi les autorisations, les sessions par cookie, le client RPC et les redirections des pages protégées.

Les 15 tests PostgreSQL comprennent trois scénarios de réservation et de collaboration, ainsi que douze scénarios de capacité et de concurrence. Deux tests MongoDB sont présents pour l'écriture, l'auteur,

l'horodatage, l'ordre et le filtrage des audits. Deux parcours Playwright couvrent la réservation suivie de son annulation et le parcours d'invitation jusqu'au check-in.

La section 9 relie les tests aux règles métier.

1.3.2. Préparer et documenter le déploiement d'une application

J'ai préparé deux Dockerfiles multi-stage et le fichier docker-compose.yml pour lancer les bases, le backend et le frontend. Compose attend que PostgreSQL et MongoDB soient prêts. Le backend applique alors les migrations avant de démarrer, et le frontend attend à son tour que l'API soit disponible. Un profil optionnel permet de charger les données de démonstration.

Le README.md décrit la configuration, le lancement, les contrôles de santé, les tests d'intégration, les sauvegardes, la restauration et le retour à une version précédente. Les principales versions applicatives sont épinglées et les secrets restent hors du dépôt. Le déploiement public utilise aussi une image Caddy 2-alpine.

Le 22 septembre 2026, j'ai choisi une machine Ubuntu mise à disposition par un ami, dans /opt/tempo/.

1.3.3. Contribuer à la mise en production dans une démarche DevOps

Le workflow .github/workflows/ci.yml contient deux jobs. Le premier vérifie le format, le lint, les types, les tests et le build. Le second construit la stack Docker Compose, attend le démarrage des services et charge le seed. Il lance ensuite les intégrations PostgreSQL et MongoDB et les deux parcours Playwright. En cas d'échec d'un parcours E2E, les traces, captures et vidéos restent disponibles dans les artefacts pendant sept jours.

La CI n° 78 du 23 septembre 2026 a réussi sur le commit 8d0d1cd, qui contient les corrections mobiles. La capture ci-dessous montre les deux jobs terminés avec succès : Quality & Tests et Docker Build Check. Elle correspond à cette version du code. [Consulter cette exécution.](#)

Tempo CI Pipeline

docs: improve mobile responsiveness with reduced padding and vertical... #78

Summary

All jobs

Quality & Tests

Docker Build Check

Run details

Usage

Workflow file

Triggered via push yesterday

Vaalley pushed · 8d0d1cd · main

Status: Success

Total duration: 2m 45s

Artifacts: -

ci.yml on: push

Quality & Tests 40s

Docker Build Check 1m 57s

Quality & Tests summary

Vitest Test Report

Summary

- Test Files: 4 passes - 4 total
- Test Results: 22 passes - 22 total

Job summary generated at run-time

GitHub Actions n° 78, commit 8d0d1cd, 23 septembre 2026 : les deux jobs ont réussi.

2. CAHIER DES CHARGES

Le cahier des charges décrit le besoin du point de vue métier. Les choix d'implémentation sont détaillés dans les sections 5 et 6.

2.1. Description de l'existant

Tempo est un projet personnel de certification, réalisé en dehors du temps en entreprise.

Je suis parti du cas d'une équipe qui partage ses bureaux sans outil dédié. Un tableur ou un planning permet de noter les réservations, mais il gère mal les demandes simultanées, les invitations et la présence réelle. Tempo regroupe donc la réservation d'un bureau ou d'une salle, l'invitation des participants et le suivi de l'occupation.

2.2. Reprise de l'existant

J'ai commencé Tempo sans reprendre de code, d'hébergement, de nom de domaine ou de documentation existants. J'ai produit le cahier des charges SPECS.md, les diagrammes, le code et la documentation au cours de l'année 2026.

2.3. Principes de référencement

Tempo est un outil interne dont l'accès demande une connexion. Ses pages n'ont pas à apparaître dans les résultats des moteurs de recherche : le cahier des charges prévoit donc de bloquer leur indexation.

2.4. Exigences de performances et de volumétrie

Les cibles de départ sont des entreprises de 50+ collaborateurs ou des ecoles/centre de formation, sur un ou plusieurs sites.

Les hypothèses de dimensionnement sont les suivantes :

- quelques dizaines d'utilisateurs connectés en même temps ;
- quelques centaines de réservations créées chaque jour ;
- un service disponible pendant les heures ouvrées ;
- un temps de réponse visé inférieur à 300 ms pour la consultation et la réservation.

Ces chiffres sont des objectifs de départ. La V1 utilise un seul backend et PostgreSQL ; à ce stade, une architecture distribuée ne me semble pas nécessaire.

2.5. Multilinguisme et adaptations pour un public spécifique

La V1 est disponible uniquement en français. Aucune traduction n'est prévue dans le périmètre actuel.

2.6. Description graphique et ergonomique

2.6.1. Composants de la charte graphique

J'ai gardé une présentation simple pour cet outil interne. Le nom Tempo sert de logo et la police est celle du système. Les composants utilisent les mêmes tailles, espacements et états sur les différents écrans.

Les couleurs viennent des variables CSS de shadcn-svelte et de Tailwind CSS : des tons neutres, une couleur principale et des couleurs pour les alertes ou les suppressions. Je réutilise les mêmes boutons, cartes, tableaux, badges, champs et messages d'erreur sur les pages.

2.6.2. Responsive design et ergonomie

J'utilise les classes adaptatives de Tailwind CSS pour ajuster les écrans à la largeur disponible.

L'administration et les tableaux sont surtout prévus pour un ordinateur. En revanche, réserver, répondre à une invitation et faire son check-in doivent aussi être possibles sur mobile.

Sur mobile, la navigation passe sous le titre et ses liens reviennent à la ligne. Les adresses de compte trop longues peuvent être coupées, et le nom de l'espace sélectionné est tronqué si besoin. J'ai réduit les marges et disposé les champs de réservation en colonne. Les tableaux gardent leur propre défilement horizontal pour ne pas élargir toute la page. Le contrôle du 24 septembre 2026, dans une fenêtre de 390 × 844 pixels, n'a montré aucun débordement de la page sur l'accueil, les réservations, les espaces, le check-in et l'audit.

Toutes les réservations

Supervisez les réservations de l'ensemble des utilisateurs

← Accueil Espaces Audit

admin@tempo.local [→]

Nouvelle Réservation

Choisir un espace

Privée

dd----yyyy

09:00

dd----yyyy

10:00

Réserver

2 réservations

Espace	Propriétaire	Dé
 Salle Atlas Salle - 6 pers.	admin@tempo.local ADMIN	15/
 RECETTE-2026-09-24 Bureau - 4 pers.	admin@tempo.local ADMIN	24/

Réservations à 390 pixels de largeur. Le tableau défile dans sa propre zone.

2.7. Besoins fonctionnels métier

2.7.1. Utilisateurs du projet

Deux profils utilisent Tempo :

- le collaborateur **USER** crée un compte, se connecte, consulte les espaces et gère ses réservations. Il peut inviter des personnes, répondre à une invitation, rejoindre une réservation publique et confirmer sa présence ;
- l'administrateur **ADMIN** possède les mêmes fonctions et gère aussi les utilisateurs et les espaces. Il voit toutes les réservations, peut les annuler et consulte les statistiques ainsi que les audits.

Le processus concerné est la gestion des espaces de travail, généralement suivie par les fonctions RH ou Office Management.

2.7.2. Informations relatives aux contenus

Tempo ne publie aucun contenu éditorial. L'application traite :

- les comptes, avec l'adresse électronique, le hash du mot de passe et le rôle ;
- les espaces, avec leur nom, leur type et leur capacité ;
- les réservations, avec le créneau, la visibilité et le propriétaire ;
- les participants, avec leur rôle, leur réponse à l'invitation et leur heure éventuelle de check-in ;
- les hashes des jetons QR et leur date d'expiration ;
- les audits de suppression, avec l'entité concernée et l'auteur de l'action.

Ces données comprennent des informations personnelles. Les mots de passe sont donc hachés, les données reçues sont vérifiées et les actions sensibles demandent une autorisation. Après une suppression, l'application tente d'enregistrer un audit.

2.7.3. Besoins fonctionnels

Un collaborateur doit pouvoir créer un compte, se connecter et se déconnecter. À l'inscription, il reçoit le rôle USER. Une fois connecté, il peut consulter les bureaux et les salles, puis réserver un espace sur un créneau libre. La réservation peut être publique ou privée ; elle doit concerner un espace existant et respecter des dates valides. Son propriétaire peut l'annuler, mais il ne peut pas supprimer la réservation d'un autre utilisateur.

Le propriétaire d'une réservation peut inviter des utilisateurs déjà inscrits. Chaque invité choisit d'accepter ou de refuser : personne ne peut répondre à sa place. Tant qu'il n'a pas répondu, son invitation compte dans les places occupées. Un utilisateur connecté peut aussi rejoindre une réservation publique, à condition qu'il reste de la place. Une fois sa participation acceptée, il peut confirmer sa présence avec le QR code pendant le créneau réservé.

L'administrateur gère les espaces : lui seul peut en créer, les modifier ou les supprimer. Chaque espace doit avoir une capacité entière d'au moins une place. Il peut également annuler une réservation et y inviter un utilisateur. Il dispose d'une vue sur les réservations, les statistiques et les suppressions enregistrées, présentées de la plus récente à la plus ancienne. L'accès à ces fonctions est réservé au rôle ADMIN.

La V1 ne prévoit pas de filtrage avancé des espaces, de gestion de plusieurs sites, de notifications ou de quotas plus détaillés.

2.8. Budget

Tempo est un projet personnel que j'ai réalisé en parallèle de mon alternance, sans budget financier attribué. J'y ai consacré du temps pour définir le besoin, concevoir l'application, développer le backend et le frontend, écrire les tests, préparer les conteneurs et rédiger les documents.

Aucun relevé horaire exhaustif n'a été tenu.

3. PRÉSENTATION DE L'ENTREPRISE ET DU SERVICE

3.1. Présentation de l'entreprise et du service

Tempo est un projet personnel de gestion d'espaces en flex-office, réalisé seul pour le titre CDA. Je l'ai développé pour travailler les compétences du titre, sans répondre à la demande d'une entreprise existante.

J'ai imaginé Tempo comme un SaaS pour des PME. J'en ai défini le besoin et développé la solution dans le cadre de la certification ; il ne s'agit pas d'une commande commerciale.

3.2. Objectifs du projet

Tempo permet à un collaborateur de réserver un bureau ou une salle, seul ou avec d'autres participants. L'administrateur gère les espaces et les comptes, consulte les réservations et suit l'occupation. Il peut aussi retrouver les suppressions enregistrées dans le journal d'audit.

Un participant accepté peut utiliser le QR code pour confirmer sa présence pendant la réservation. Comme il peut transmettre ce code à quelqu'un d'autre, le check-in ne prouve pas à lui seul qu'il se trouve sur place.

3.3. Cible adressée par le projet

Tempo s'adresse aux entreprises de plus de 50 collaborateurs, ainsi qu'aux écoles et centres de formation qui partagent des bureaux ou des salles. Un utilisateur doit pouvoir réserver rapidement ; la personne qui gère les espaces doit pouvoir les administrer et voir les réservations en cours.

3.4. Processus utilisateur impacté

Avec Tempo, les réservations sont suivies dans l'application plutôt que dans un tableur ou un planning partagé. Les équipes RH et les responsables des espaces peuvent consulter l'occupation et voir quels espaces sont les plus utilisés.

4. GESTION DE PROJET

Le suivi repose sur Trello, Git et les documents de travail du dépôt.

4.1. Intervenants sur le projet

J'ai réalisé Tempo seul. J'ai donc pris en charge :

- l'expression du besoin et le cahier des charges ;
- la conception des données, de l'architecture, des diagrammes et des écrans ;
- le développement du backend, du frontend et des migrations ;
- la rédaction et l'exécution des tests ;
- Docker Compose, GitHub Actions et la documentation de déploiement.

Comme j'ai réalisé le projet seul, je n'avais ni client, ni chef de projet, ni designer extérieur. J'ai noté mes choix dans les documents du dépôt et dans le suivi des tâches.

4.2. Méthodologie

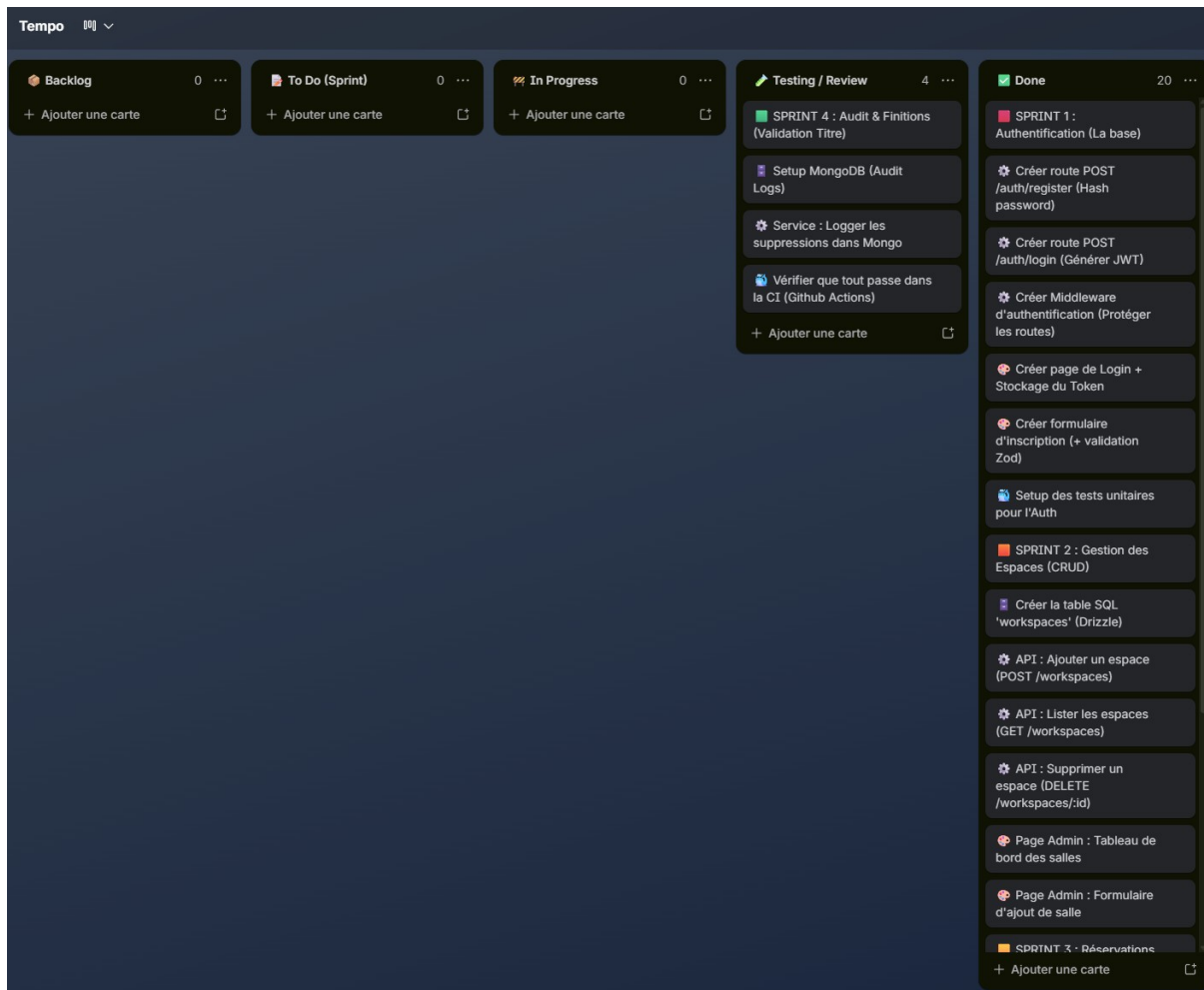
Dans Trello, je déplaçais les cartes entre trois colonnes : « À faire », « En cours » et « Terminé ». Une carte correspondait à une fonctionnalité ou à une tâche technique assez précise pour être vérifiée séparément.

Je testais chaque fonctionnalité avant de passer à la suivante. J'ai complété les premiers parcours avec la visibilité publique ou privée, les invitations et le check-in. Le plan de corrections m'a ensuite servi à reprendre les problèmes de sécurité et les points manquants dans la documentation et le déploiement.

4.3. Outils, planning et suivi

Après avoir défini le besoin et conçu l'application, j'ai développé le backend et le frontend en ajoutant les tests au fur et à mesure. J'ai ensuite préparé les conteneurs Docker, la CI et la documentation.

Je suivais les tâches dans Trello et les changements de code dans GitHub. Les corrections à faire sont regroupées dans les documents de travail du dépôt. À chaque envoi de code, GitHub Actions lance les contrôles.



Add class diagram documentation for domain model and business logic	63460ef	<>
Remove Playwright browser testing setup and simplify test configuration	89e263e	<>
test: Change fetch mock from global to window in page.svelte.spec.ts	68f2524	<>
format	39d1de1	<>
test: Add fetch mock to page.svelte.spec.ts	6b4f770	<>
fix: Playwright browser installation step to CI workflow	d8b7306	<>
Initial commit	6bfaa53	<>

4.4. Objectifs de qualité

Pour vérifier le projet, je m'appuie sur les tests et les contrôles automatiques suivants :

- l'inventaire comprend 106 tests backend unitaires et HTTP, 15 tests PostgreSQL, 2 tests MongoDB, 22 tests frontend et 2 parcours E2E ; les exécutions réellement constatées sont précisées en section 9 ;
- Oxlint, Oxfmt, TypeScript et Svelte Check contrôlent le code avant le build ;

- Zod valide les entrées, les mots de passe sont hachés, les routes sont protégées par JWT et les droits sont vérifiés côté API ;
- les modules séparent les routes, les services et la persistance ;
- GitHub Actions reconstruit l'application et exécute la recette Docker sur un environnement neuf.

Le workflow GitHub Actions lance ces vérifications. La protection de la branche, elle, se règle dans les paramètres du dépôt GitHub.

5. SPÉCIFICATIONS FONCTIONNELLES

Cette partie décrit la V1. J'ai gardé les diagrammes de départ ; les sections correspondantes expliquent les différences avec le code actuel.

5.1. Contraintes du projet et livrables attendus

5.1.1. Criticité de l'application

Si Tempo est indisponible pendant la journée, les utilisateurs ne peuvent plus consulter ou créer de réservation, répondre aux invitations ni confirmer leur présence. Pour cet outil interne, j'ai retenu une criticité modérée.

L'application vise une utilisation de 8 h à 19 h, du lundi au vendredi, par quelques dizaines à quelques centaines d'utilisateurs.

5.1.2. Applications connexes

Tempo fonctionne sans annuaire, calendrier ou logiciel RH externe. Une connexion SSO ou une synchronisation avec un calendrier pourrait être ajoutée plus tard.

5.1.3. Services tiers

L'application n'utilise ni service d'emailing, ni CRM, ni outil d'analytics externe. GitHub Actions est utilisé uniquement pour la CI. La stack de démonstration reste exécutable en local avec Docker Compose.

5.1.4. Livrables attendus

Les livrables sont :

- le cahier des charges SPECS.md ;
- les modèles MERISE, les diagrammes UML et la maquette Figma ;
- le backend, le frontend et les migrations versionnés sur GitHub ;
- les tests unitaires, HTTP, frontend, d'intégration et E2E ;
- les Dockerfiles et le fichier docker-compose.yml ;
- le workflow GitHub Actions ;
- le seed de démonstration, le README.md et le présent dossier.

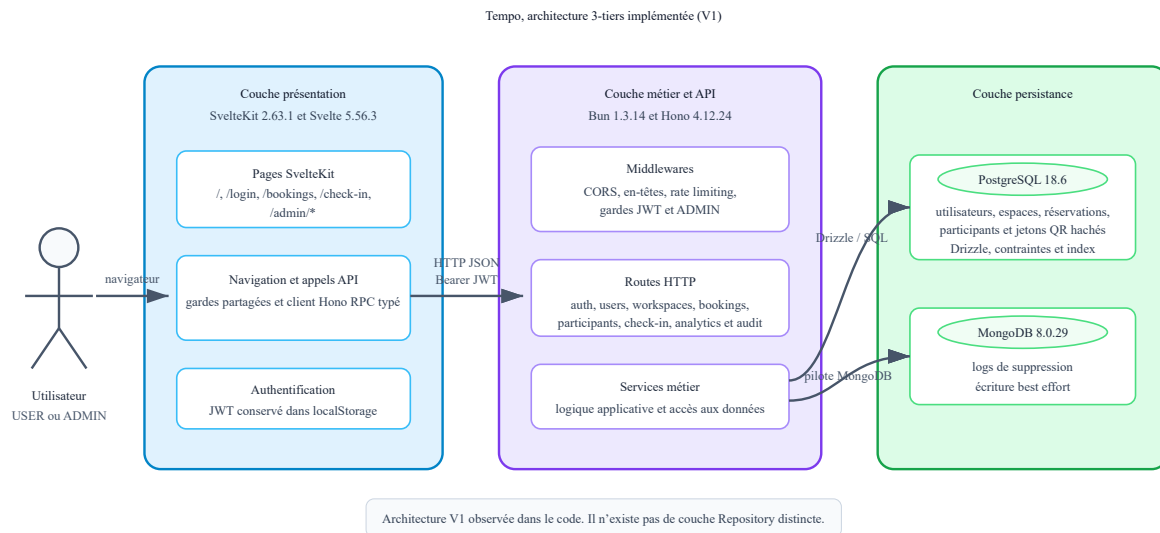
5.2. Architecture logicielle du projet

Tempo suit une architecture en trois couches dans un monorepo Bun.

Les pages sont écrites avec SvelteKit et Svelte 5. Elles appellent le backend avec un client Hono RPC qui reprend les types. À la réception d'une requête, Hono passe par les middlewares, puis appelle le service concerné dans auth, users, workspaces, bookings, analytics ou audit.

PostgreSQL stocke les comptes, les espaces, les réservations, les participants et les hashes des jetons QR. Les audits de suppression sont dans MongoDB. Une requête suit généralement le chemin Route -> Service -> Drizzle/Mongo : je n'ai pas ajouté de couche Repository séparée.

Le frontend et le backend disposent chacun d'un Dockerfile multi-stage. Docker Compose les lance avec PostgreSQL et MongoDB.



Sur ce diagramme, le JWT est encore stocké dans localStorage et envoyé avec un en-tête Bearer. Dans le code actuel, il passe par un cookie HttpOnly ; le client utilise credentials: include et les requêtes sont protégées contre le CSRF. Ce changement ne modifie pas les trois couches de l'architecture.

5.3. Maquettes et enchaînement des écrans

5.3.1. Cartographie

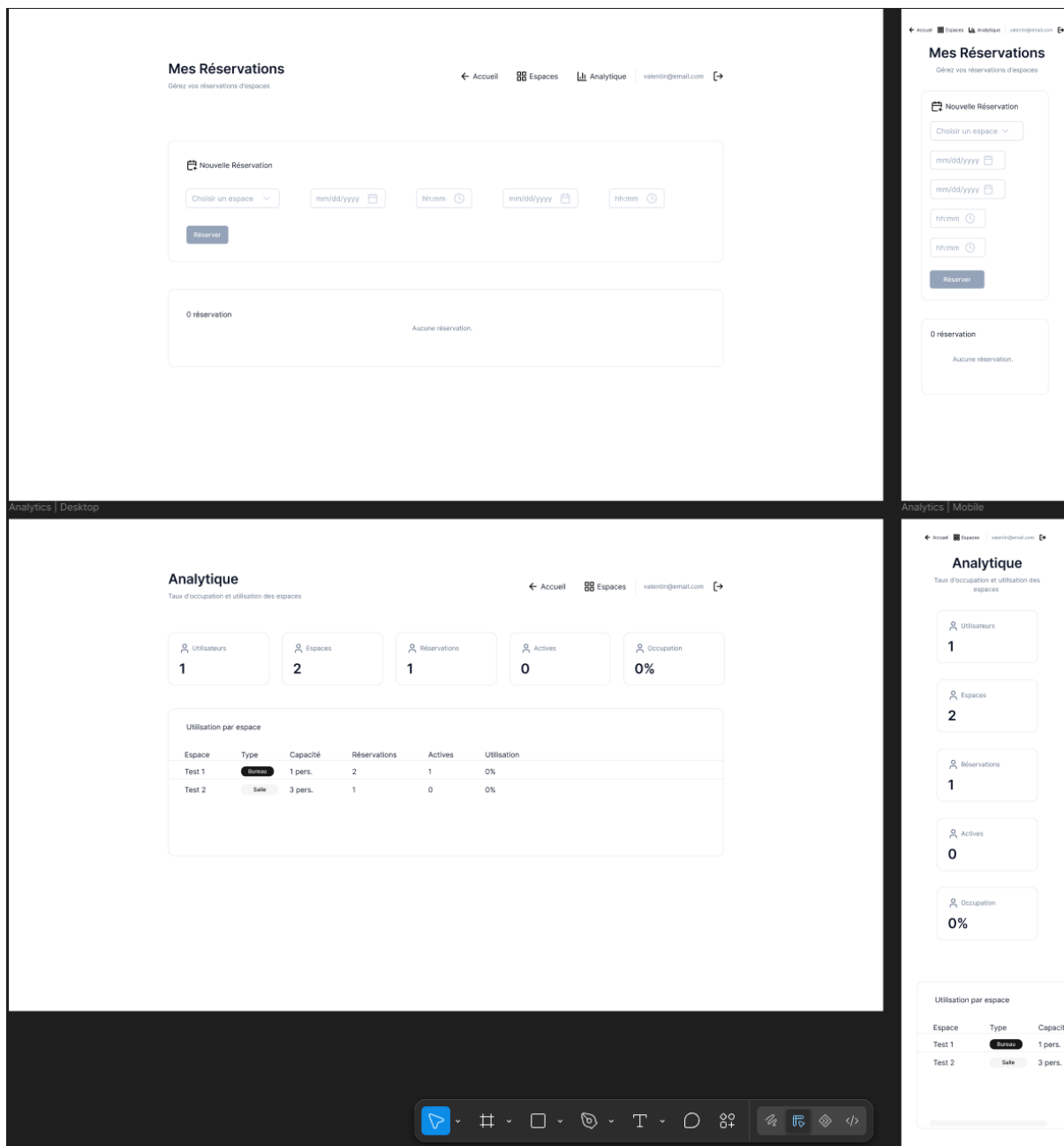
/	Accueil après authentification
/login	Connexion et inscription
/bookings	Réservations, invitations, participants et QR code
/check-in	Validation de présence depuis un QR code
/admin/workspaces	Gestion des espaces, rôle ADMIN
/admin/analytics	Statistiques d'occupation, rôle ADMIN
/admin/audit	Consultation des audits, rôle ADMIN

Un visiteur qui tente d'ouvrir une page protégée est redirigé vers /login. Après la connexion, l'accueil affiche les fonctions disponibles pour son rôle. Le collaborateur accède à ses réservations, à ses invitations et aux réservations publiques. L'administrateur voit aussi les outils de gestion, les statistiques, les audits et l'ensemble des réservations.

Lorsqu'un utilisateur non connecté scanne un QR code, la destination est conservée dans sessionStorage pendant le passage par la page de connexion. Le jeton reste dans le fragment de l'URL et n'est pas envoyé dans la requête initiale au serveur frontend.

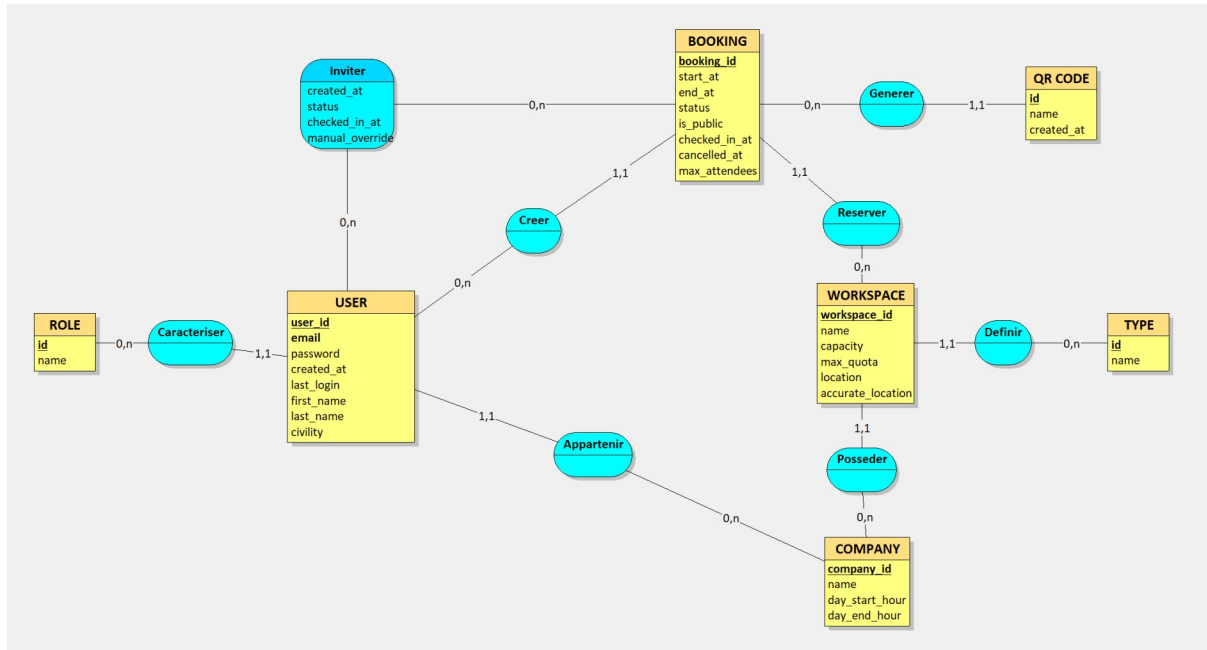
5.3.2. Maquettes

La [maquette Figma](#) m'a servi à préparer la navigation et les premiers écrans. L'interface a ensuite évolué avec shadcn-svelte et l'ajout de fonctionnalités. Les captures montrent la version développée.

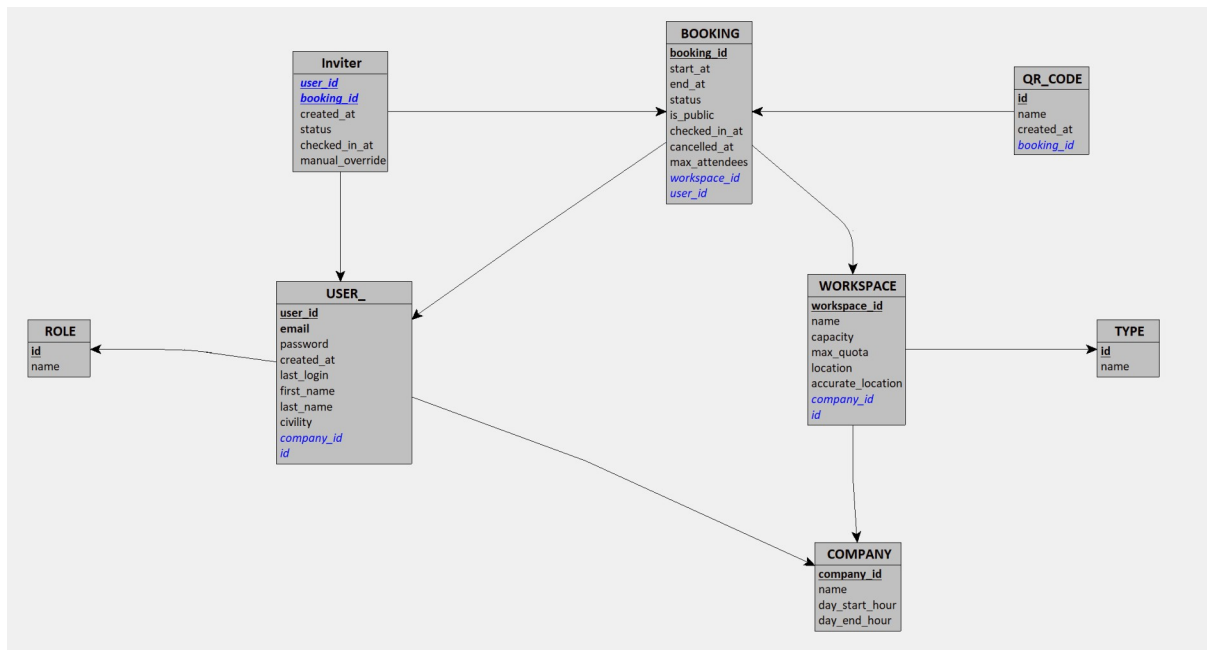


5.4. Modélisation des données

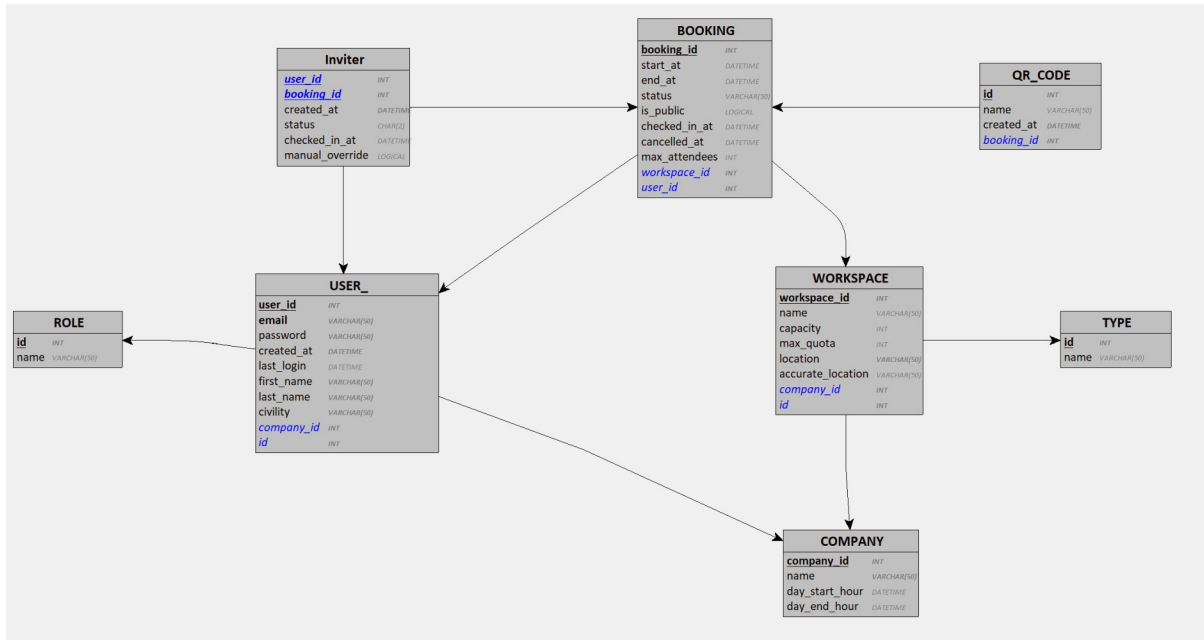
5.4.1. MCD (MERISE)



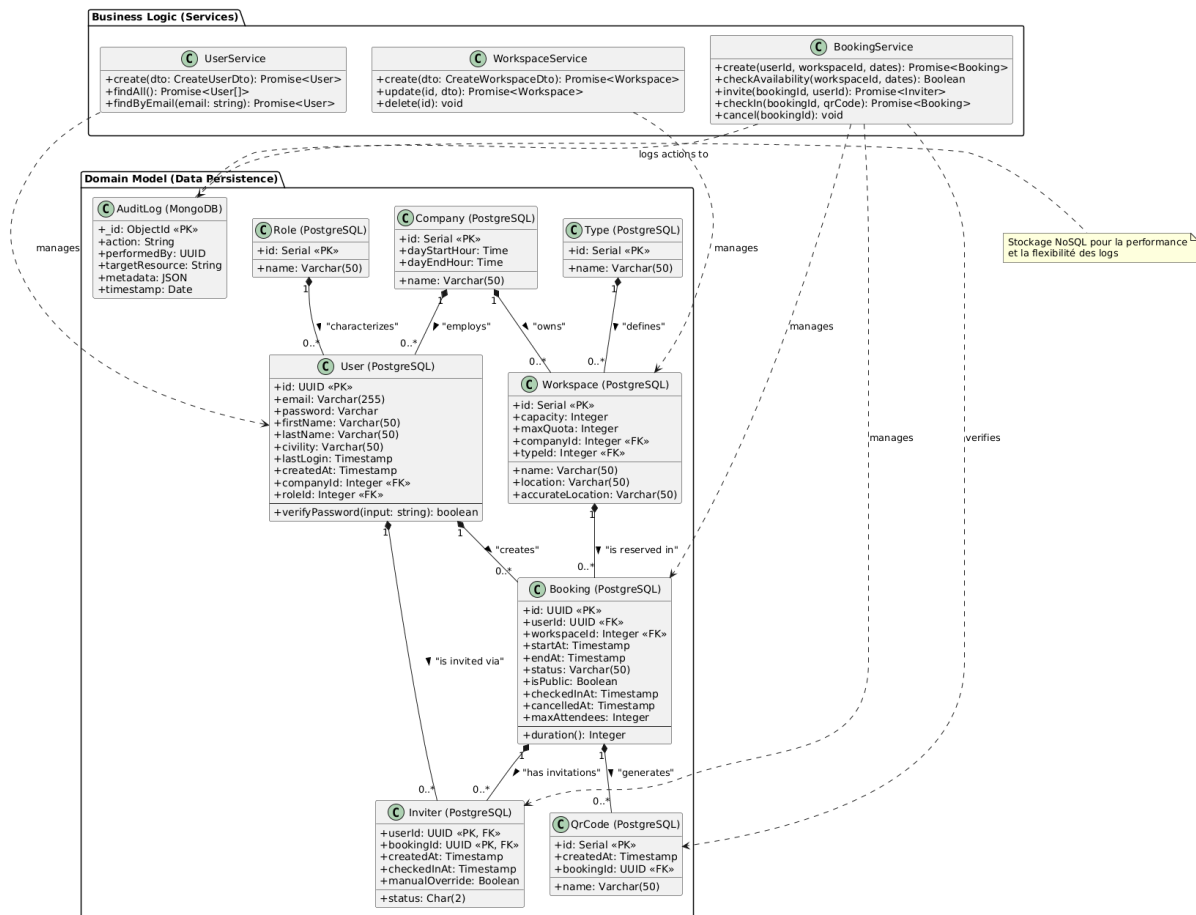
5.4.2. MLD (MERISE)



5.4.3. MPD (MERISE)



5.4.4. Diagramme de classes (UML)



Les modèles prévoyaient des entreprises, des localisations, des quotas avancés, des notifications et une annulation logique. Ces fonctions ne sont pas présentes dans la V1, qui gère les comptes, les espaces, les réservations, les participants et les jetons QR. Les différences sont détaillées dans docs/ECARTS_DIAGRAMMES_2026-09-21.md.

Le schéma Drizzle se trouve dans apps/backend/src/db/schema.ts. Les migrations 0000 à 0004 créent les cinq tables PostgreSQL : users, workspaces, bookings, booking_participants et booking_qr_tokens. Les audits sont stockés séparément dans MongoDB.

Dans le code, les rôles et les types d'espace sont des enums PostgreSQL. Les comptes et les réservations utilisent des UUID, alors que le MPD indique des entiers. La réponse à une invitation et le check-in sont enregistrés dans booking_participants. Chaque réservation conserve au plus un hash de jeton QR actif avec sa date d'expiration, contrairement aux modèles qui prévoient plusieurs QR.

5.5. Création et modification de la base de données

5.5.1. Script de création

Drizzle Kit génère les migrations à partir du schéma TypeScript. La migration 0000_gorgeous_vapor.sql crée les premiers enums, les comptes et les espaces :

```
CREATE TYPE "public"."role" AS ENUM('ADMIN', 'USER');
CREATE TYPE "public"."workspace_type" AS ENUM('DESK', 'MEETING_ROOM');

CREATE TABLE "users" (
  "id" uuid PRIMARY KEY DEFAULT gen_random_uuid() NOT NULL,
  "email" text NOT NULL UNIQUE,
  "password" text NOT NULL,
  "role" "role" DEFAULT 'USER',
  "created_at" timestamp DEFAULT now()
);

CREATE TABLE "workspaces" (
  "id" serial PRIMARY KEY NOT NULL,
  "name" text NOT NULL,
  "type" "workspace_type" NOT NULL,
  "capacity" integer DEFAULT 1 NOT NULL,
  "created_at" timestamp DEFAULT now()
);
```

La migration 0001_optimal_black_tarantula.sql ajoute la table des réservations :

```
CREATE TABLE "bookings" (
  "id" uuid PRIMARY KEY DEFAULT gen_random_uuid() NOT NULL,
  "user_id" uuid NOT NULL,
  "workspace_id" integer NOT NULL,
  "start_at" timestamp NOT NULL,
  "end_at" timestamp NOT NULL,
  "created_at" timestamp DEFAULT now(),
  FOREIGN KEY ("user_id") REFERENCES "users"("id") ON DELETE CASCADE,
  FOREIGN KEY ("workspace_id") REFERENCES "workspaces"("id") ON DELETE CASCADE
);
```

5.5.2. Choix retenus

Avec Drizzle, je retrouve les types du schéma SQL dans les services TypeScript. Je conserve les migrations SQL dans Git et j'y ajoute les contraintes que Drizzle ne permet pas de décrire directement.

ON DELETE CASCADE supprime aussi les données qui dépendent de l'élément retiré. Cela évite de garder des réservations ou des participants sans enregistrement parent. L'audit est écrit après la suppression, dans MongoDB. Comme cette écriture ne fait pas partie de la transaction PostgreSQL, son échec n'annule pas la suppression.

5.5.3. Scripts de modification

La migration 0002_booking_overlap_constraint.sql ajoute les contraintes temporelles :

```
CREATE EXTENSION IF NOT EXISTS "btree_gist";

ALTER TABLE "bookings"
ADD CONSTRAINT "bookings_valid_time_range"
CHECK ("end_at" > "start_at");

ALTER TABLE "bookings"
ADD CONSTRAINT "bookings_workspace_time_exclusion"
EXCLUDE USING gist (
    "workspace_id" WITH =,
    tsrange("start_at", "end_at", '[') WITH &&
);
```

La migration 0003_data_integrity_indexes.sql renforce les valeurs obligatoires et les index :

```
UPDATE "users" SET "role" = 'USER' WHERE "role" IS NULL;
UPDATE "workspaces" SET "capacity" = 1 WHERE "capacity" < 1;

ALTER TABLE "users" ALTER COLUMN "role" SET NOT NULL;
ALTER TABLE "workspaces"
ADD CONSTRAINT "workspaces_capacity_check" CHECK ("capacity" >= 1);

CREATE INDEX "bookings_user_id_idx" ON "bookings" ("user_id");
CREATE INDEX "bookings_workspace_time_idx"
ON "bookings" ("workspace_id", "start_at", "end_at");
```

La migration 0004_booking-collaboration-checkin.sql ajoute la visibilité, les participants, les statuts d'invitation et les jetons QR. Pendant la migration, chaque propriétaire existant devient un participant OWNER avec le statut ACCEPTED.

5.5.4. Justification des contraintes

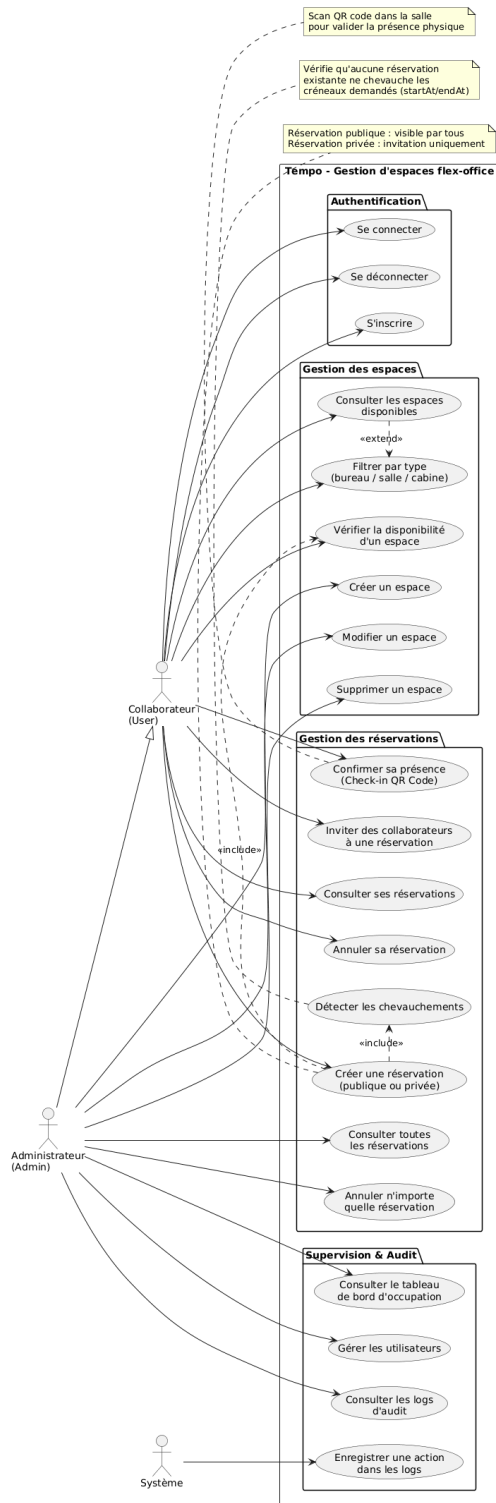
Deux requêtes peuvent vérifier en même temps qu'un espace est libre, puis tenter de le réserver. La contrainte d'exclusion GiST empêche ce doublon dans PostgreSQL. L'intervalle [] inclut le début et exclut la fin : une réservation peut donc commencer exactement quand la précédente se termine. En cas de conflit, le backend retourne BOOKING_OVERLAP avec le statut HTTP 409.

La contrainte CHECK rejette un créneau vide ou inversé. La migration 0003 rend le rôle obligatoire, impose une capacité minimale et indexe les recherches principales. La migration 0004 empêche qu'un utilisateur apparaisse deux fois dans la même réservation et indexe les recherches par réservation, utilisateur et statut.

Avant d'ajouter un participant, le service verrouille l'espace puis la réservation, et compte les places déjà occupées. Une modification de capacité utilise le même verrou sur l'espace. Elle est refusée si la nouvelle capacité ne suffit plus pour les participants acceptés ou en attente d'une réservation non terminée. Les douze tests consacrés à ces règles vérifient aussi le cas où plusieurs opérations arrivent en même temps.

5.6. Diagrammes de comportement

5.6.1. Diagramme de cas d'utilisation global (UML)



La V1 couvre l'inscription, la connexion, la consultation des espaces, les réservations publiques ou privées, les invitations, les participants, le check-in, l'administration des espaces et des utilisateurs, les statistiques et les audits. Le filtrage avancé et le multi-site ne sont pas encore implémentés.

Le rôle ADMIN reprend les droits du collaborateur et ajoute les fonctions de gestion. Un audit est tenté après la suppression d'un espace ou d'une réservation. Le service sait aussi produire un audit utilisateur, mais la V1 ne propose pas de route de suppression de compte.

5.7. Fonctionnalités détaillées les plus significatives

5.7.1. Fonctionnalité 1 : réserver un espace

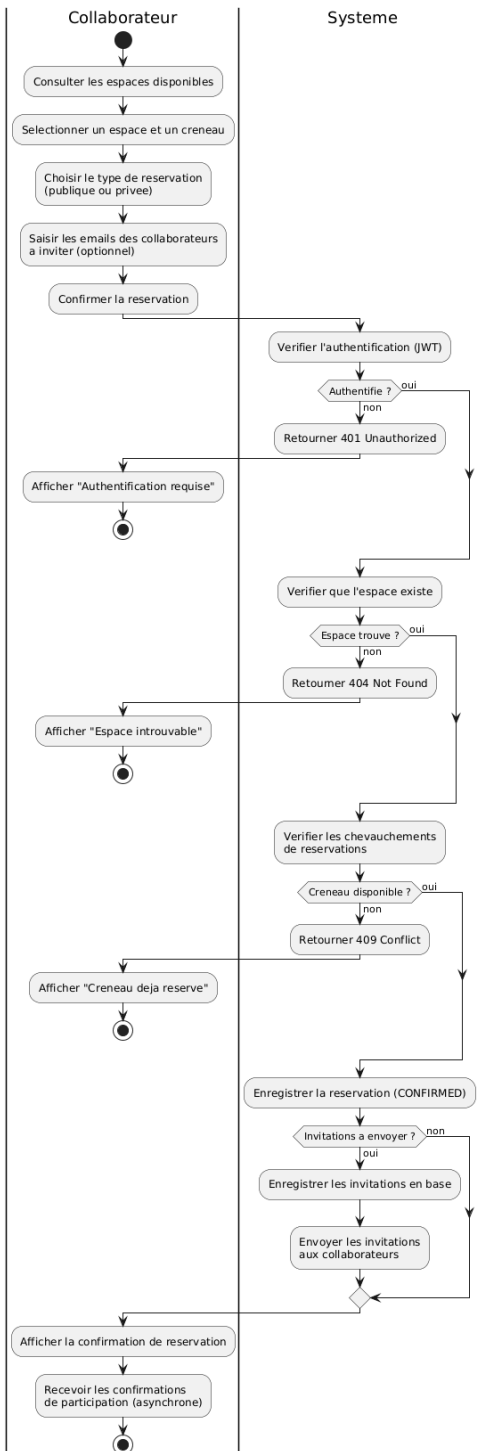
Le collaborateur choisit un espace et un créneau, puis définit la réservation comme publique ou privée.

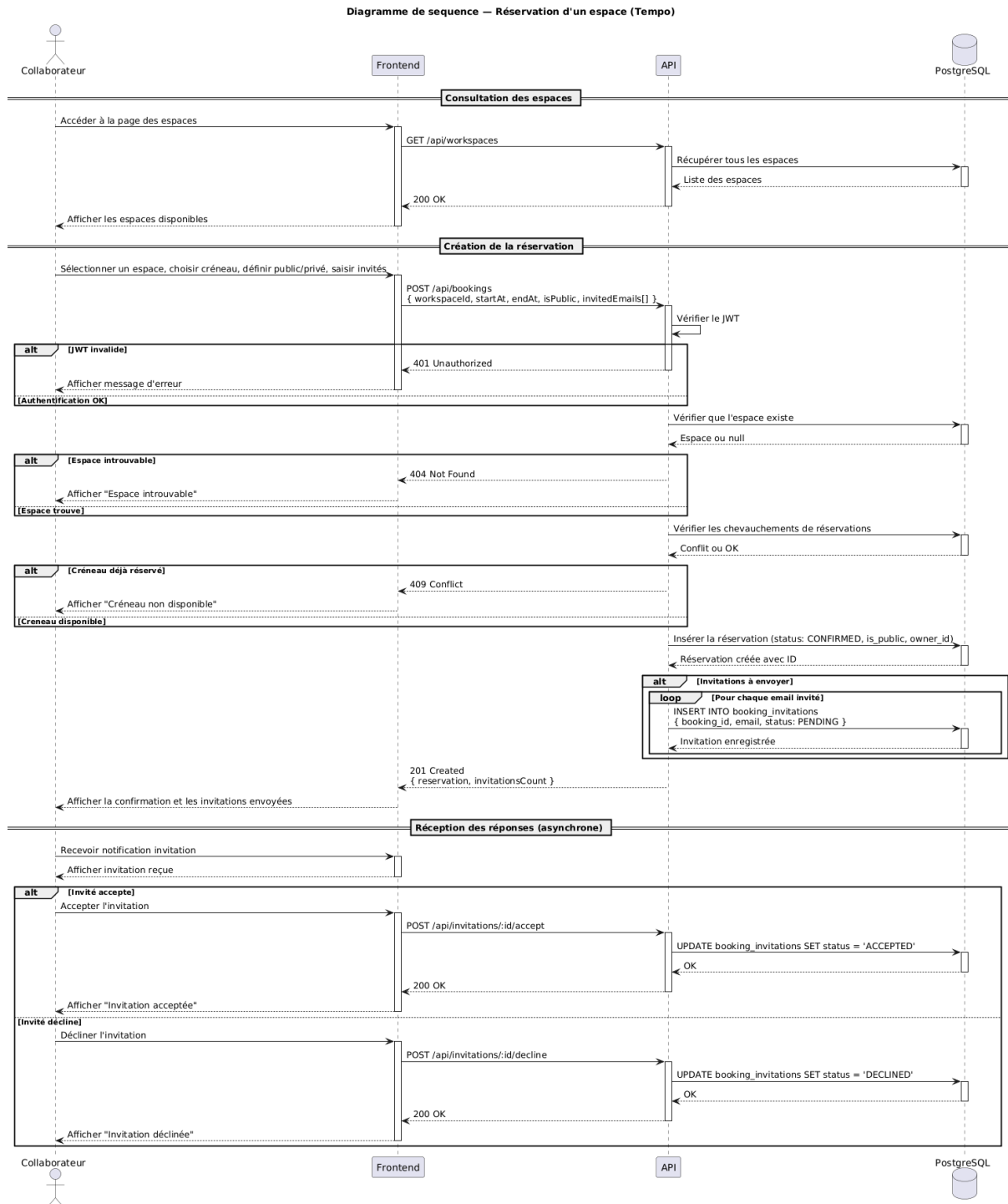
Depuis le formulaire de /bookings, un utilisateur connecté choisit un espace existant, un début, une fin et une visibilité. La requête POST /bookings transmet workspaceId, startAt, endAt et visibility. Zod valide ces données, puis le service contrôle l'espace et le créneau. La réservation et son propriétaire, enregistré comme participant, sont créés dans une transaction PostgreSQL.

La réservation apparaît ensuite dans la liste. Deux requêtes concurrentes ne peuvent pas réserver le même espace au même moment. L'API retourne HTTP 400 pour des données invalides, 401 sans session, 404 si l'espace n'existe pas et 409 en cas de chevauchement.

Tests associés : bookings.service.spec.ts, http.routes.spec.ts, postgres-bookings.integration.spec.ts, booking-flow.spec.ts, migration 0002.

Diagramme d'activité - Reservation d'un espace (Tempo)





Les diagrammes de réservation utilisent `/api/bookings`, `isPublic`, une liste d'invités à la création et une table d'invitations séparée. La route réelle est `POST /bookings` avec `visibility` ; les invitations sont créées ensuite dans `booking_participants`. La contrainte GiST garantit le non-chevauchement, même si deux précontrôles applicatifs réussissent. L'annulation reste une suppression physique.

5.7.2. Fonctionnalité 2 : annuler une réservation

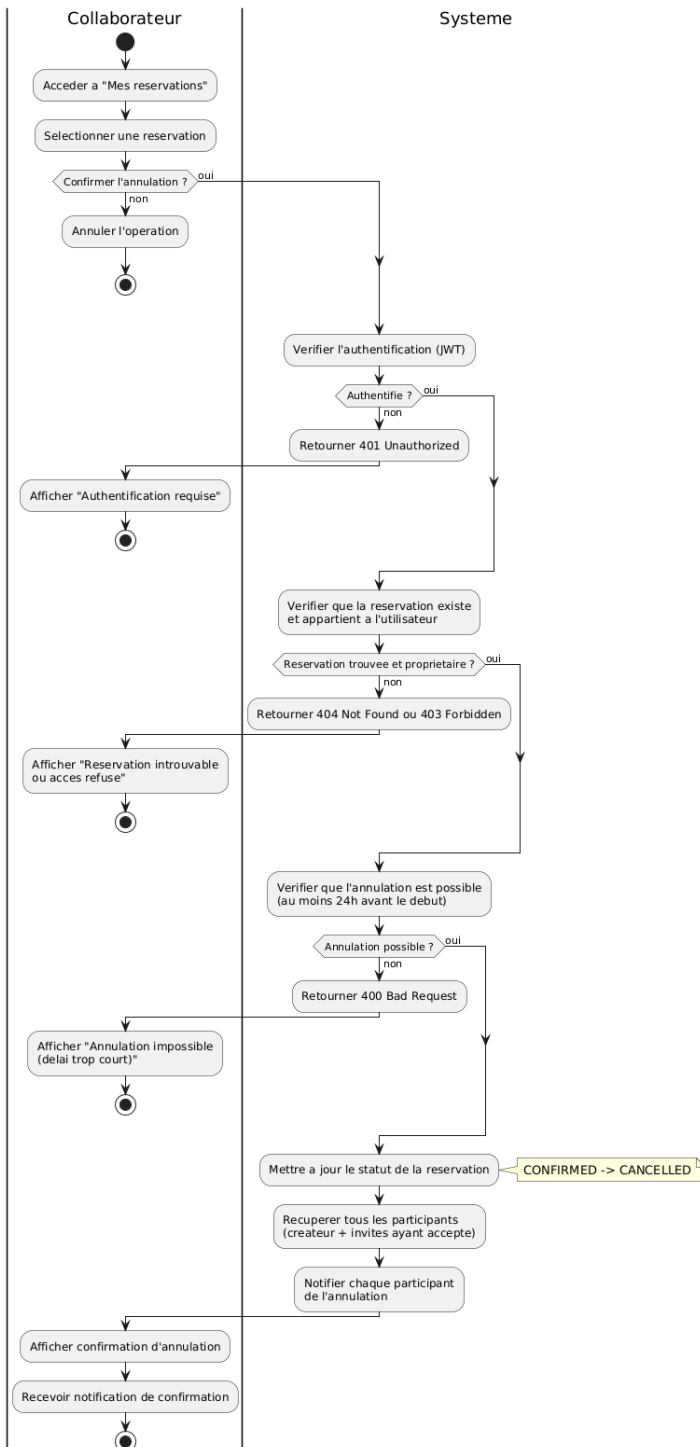
Un collaborateur peut annuler sa propre réservation. Un administrateur peut annuler celle de n'importe quel utilisateur.

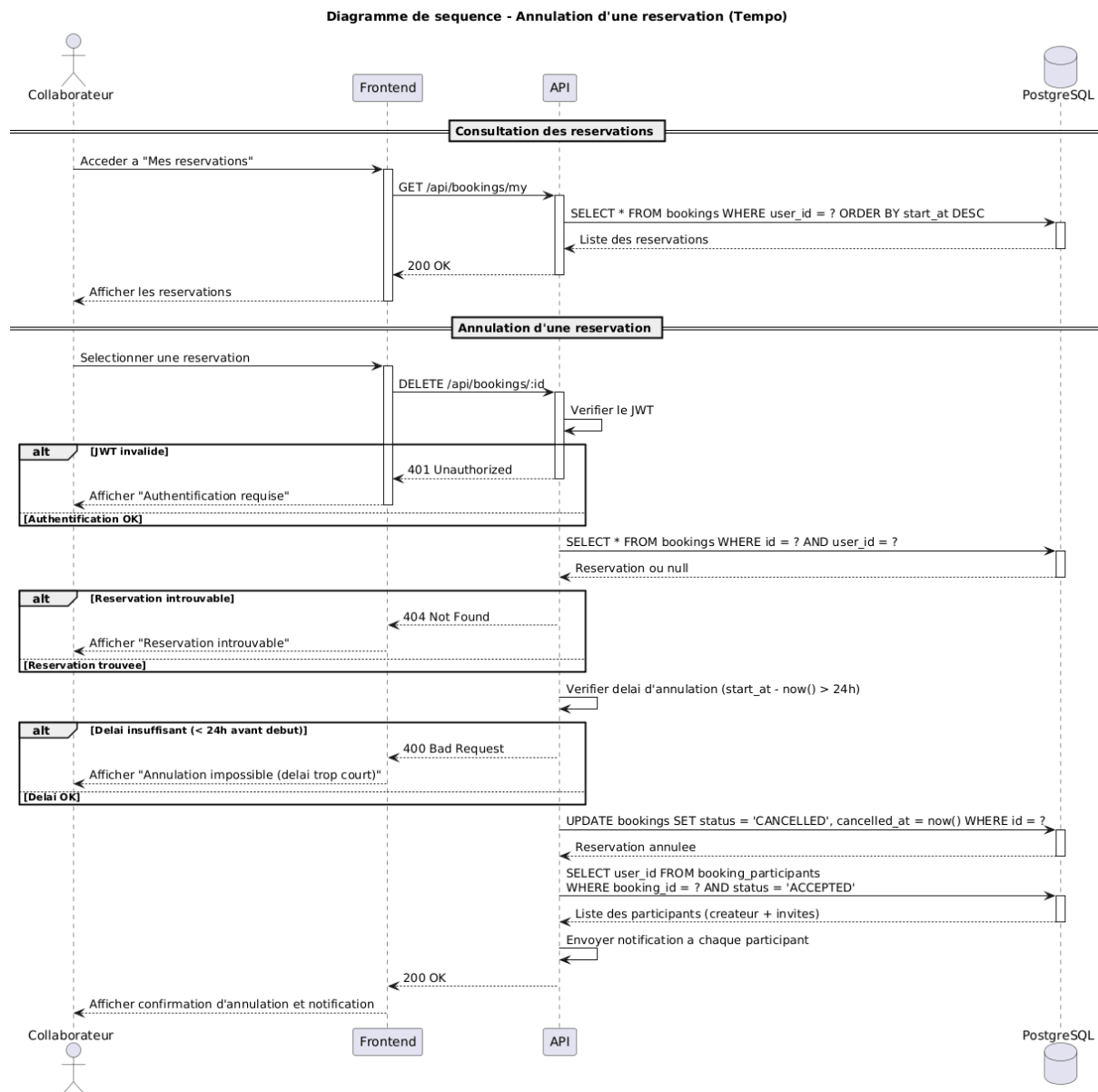
L'action « Annuler » dans /bookings appelle DELETE /bookings/:id avec l'UUID de la réservation. L'API vérifie la session, l'existence de la réservation et le droit de la supprimer. Elle effectue la suppression dans PostgreSQL, puis tente d'enregistrer l'audit dans MongoDB.

La réservation disparaît de la liste. Si MongoDB est disponible, l'audit conserve les données supprimées et l'auteur de l'action. Un identifiant invalide donne HTTP 400, une session absente 401, une réservation appartenant à un tiers 403 pour un collaborateur, et une réservation introuvable 404.

Tests associés : bookings.service.spec.ts, http.routes.spec.ts, mongo-audit.integration.spec.ts, booking-flow.spec.ts.

Diagramme d'activite - Annulation d'une reservation (Tempo)



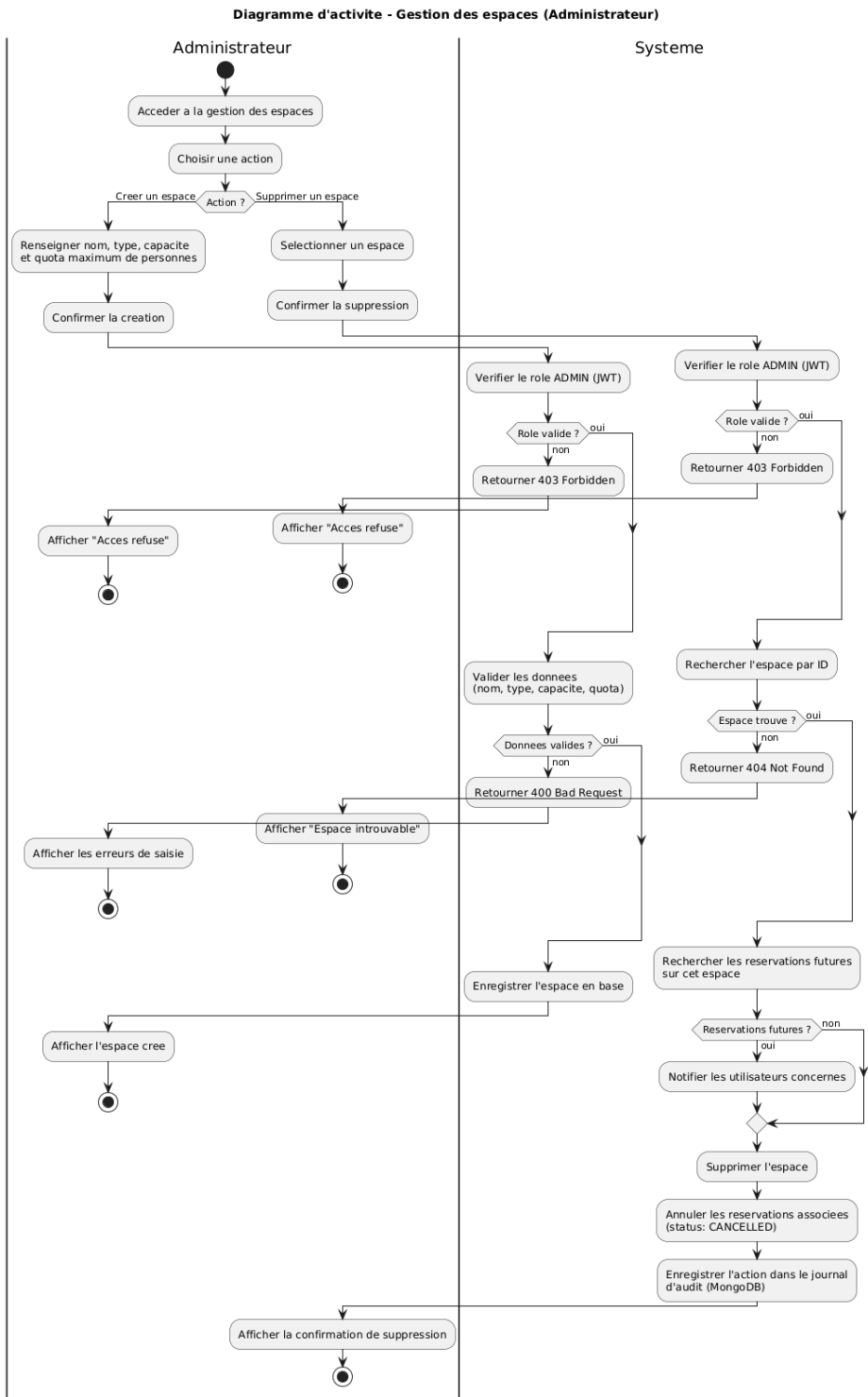


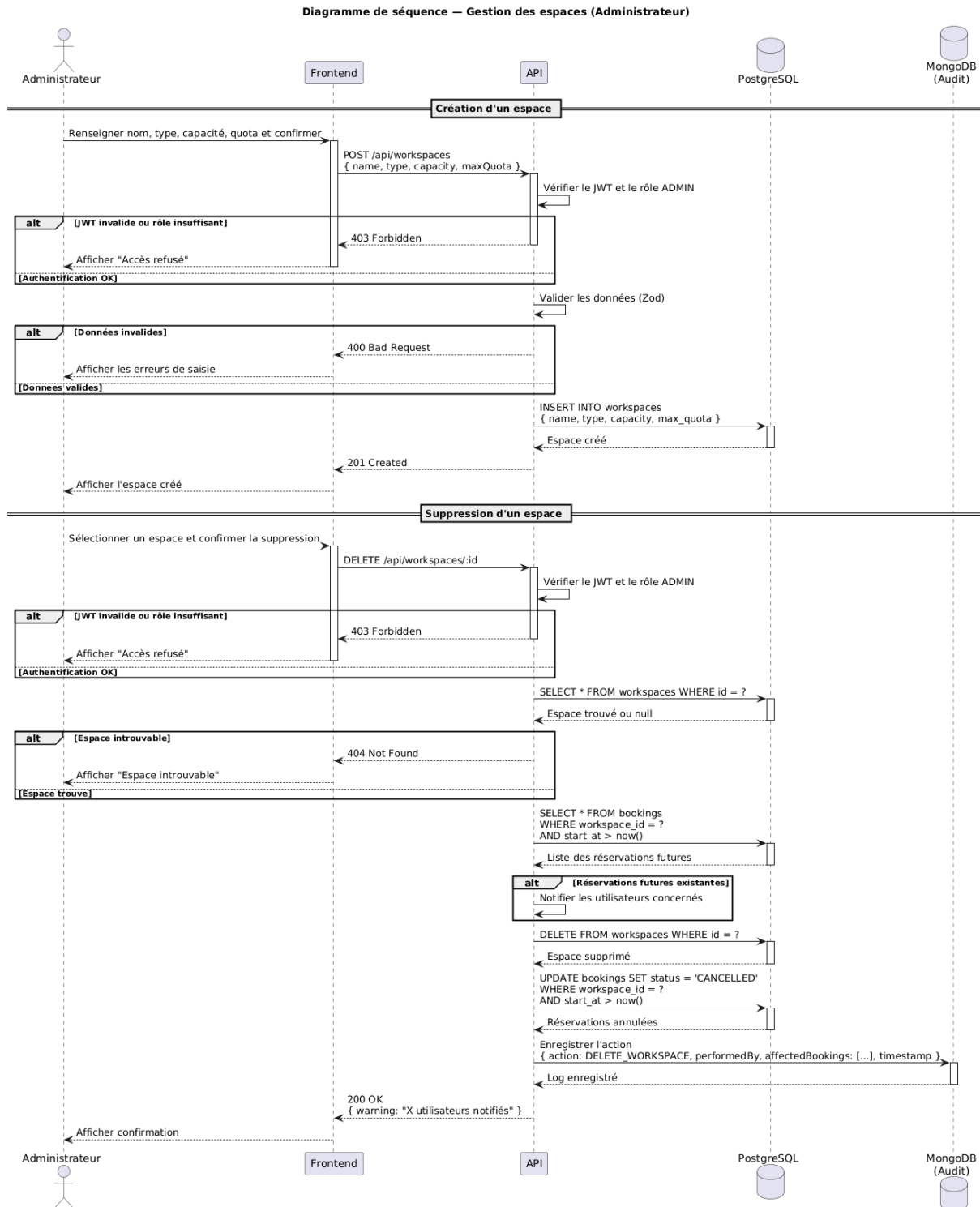
Les diagrammes d'annulation imposent un délai de 24 heures, un statut CANCELLED et des notifications. Le code ne prévoit ni ce délai ni ces notifications. Il supprime la réservation et tente d'écrire un audit MongoDB ; l'administrateur peut également annuler celle d'un tiers.

5.7.3. Fonctionnalité 3 : gérer les espaces

L'administrateur crée, modifie et supprime les espaces depuis `/admin/workspaces`. Il renseigne le nom, le type (DESK ou MEETING_ROOM) et une capacité d'au moins une place. Tous les utilisateurs connectés peuvent consulter `GET /workspaces` ; les requêtes POST, PATCH et DELETE sont réservées au rôle ADMIN. Zod valide les données avant l'écriture dans PostgreSQL. Le tableau est actualisé après l'opération et une suppression déclenche une tentative d'audit. L'API retourne HTTP 400 pour des données invalides, 401 sans session, 403 si un utilisateur standard tente une modification et 404 si l'espace n'existe pas.

Tests associés : `workspaces.dto.spec.ts`, `workspaces.service.spec.ts`, `admin.routes.spec.ts`, `http.routes.spec.ts`.





Les diagrammes de gestion des espaces ajoutent un quota et des notifications, absents du code. Une suppression entraîne les cascades PostgreSQL, pas une mise à jour vers CANCELLED. La modification par PATCH et le contrôle des capacités sont présents dans la V1 mais non détaillés dans ces séquences.

5.7.4. Fonctionnalité 4 : effectuer un check-in par QR code

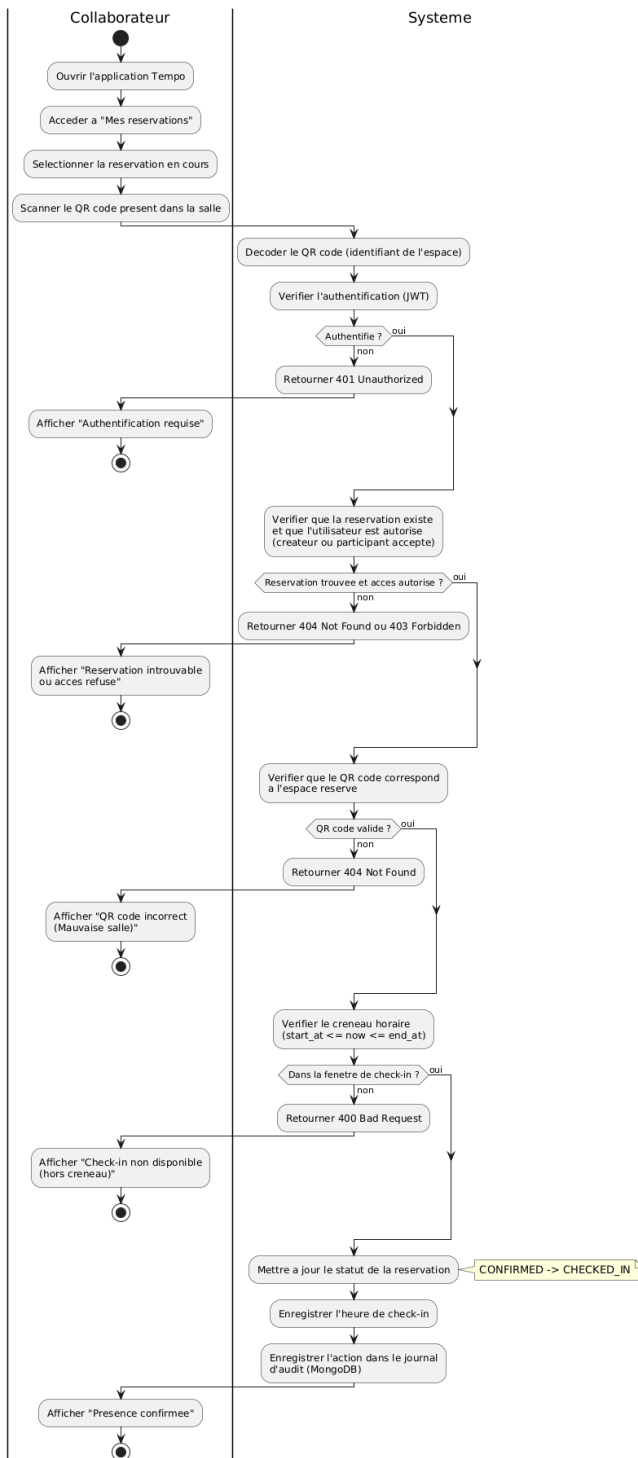
Le propriétaire ou un administrateur génère le QR code. Un participant accepté le scanne pendant le créneau.

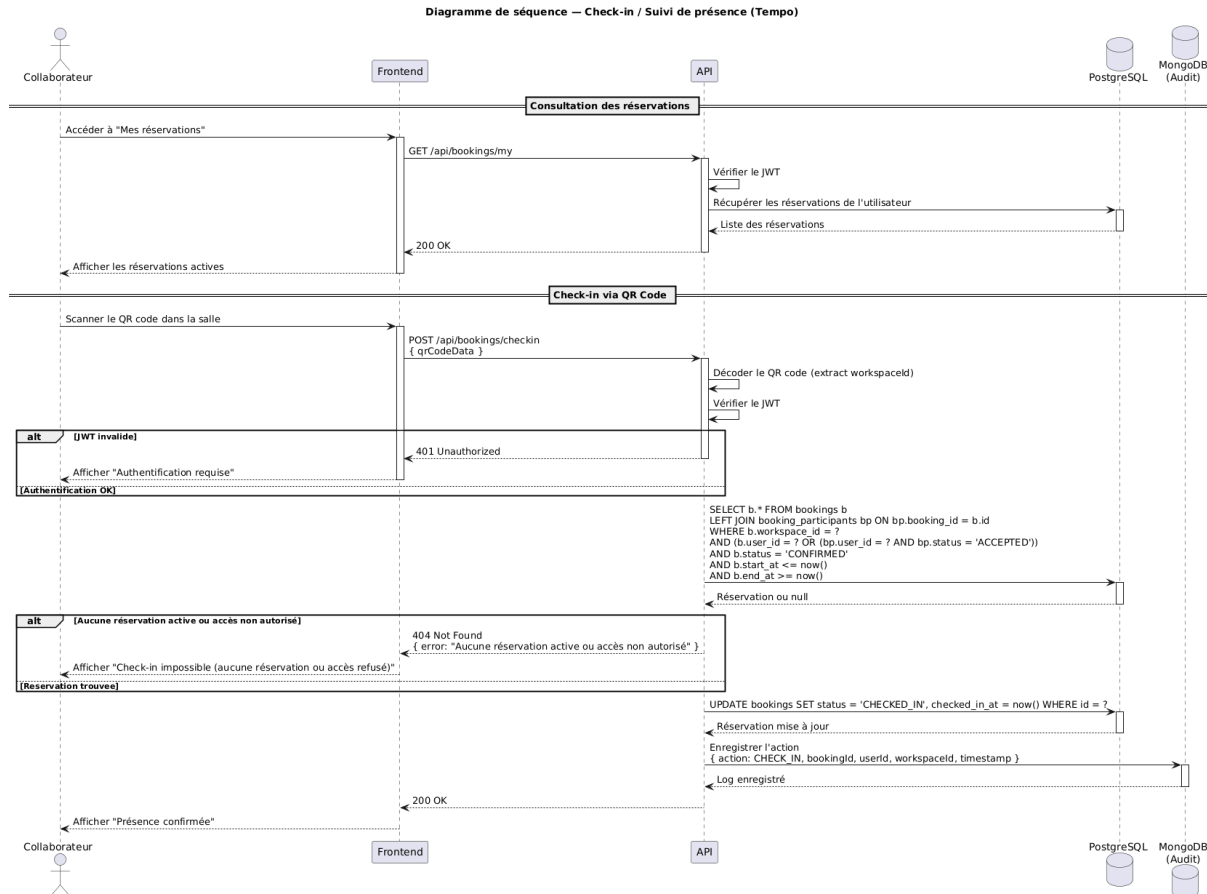
Le QR est généré dans `/bookings` par `POST /bookings/:id/qr`. Son lien ouvre `/check-in`, qui transmet l'identifiant de réservation et le jeton à `POST /bookings/:id/check-in`. Le serveur conserve le hash du jeton et vérifie la participation, le créneau et la validité du jeton avant d'enregistrer `checkedInAt`.

Un jeton invalide ou un utilisateur non autorisé donne HTTP 403. L'API retourne 409 si l'invitation n'est pas acceptée ou si le créneau n'est pas en cours. Le check-in concerne uniquement cette réservation et doit avoir lieu pendant son créneau.

Tests associés : `booking-collaboration.routes.spec.ts`, `postgres-bookings.integration.spec.ts`, `route-guard.spec.ts`, second parcours `booking-flow.spec.ts`.

Diagramme d'activité - Check-in / Suivi de presence (Tempo)





Les diagrammes de check-in montrent un QR lié à l'espace, un statut global CHECKED_IN et un audit MongoDB. La V1 utilise un jeton propre à la réservation, enregistre checkedInAt sur le participant et ne produit pas d'audit de check-in. Sa fenêtre est startAt <= maintenant < endAt : la borne de fin est exclue.

5.7.5. Fonctionnalité 5 : s'authentifier

La page /login permet de s'inscrire et de se connecter avec une adresse email et un mot de passe. Elle appelle POST /auth/register ou POST /auth/login. L'inscription est ouverte ; la connexion nécessite un compte existant. Après validation par Zod, Bun.password hache le mot de passe à l'inscription ou le vérifie à la connexion. La connexion produit un JWT valable 24 heures.

L'API ne renvoie jamais le hash du mot de passe. Le rôle contenu dans le JWT sert au contrôle des routes protégées. Les erreurs sont HTTP 400 pour des données invalides, 401 pour de mauvais identifiants, 409 si l'adresse est déjà utilisée et 429 lorsque la limite de requêtes est atteinte.

Tests associés : auth.service.spec.ts, app.security.spec.ts, rate-limit.spec.ts, auth.svelte.spec.ts, booking-flow.spec.ts.

Le JWT est transmis dans le cookie tempo_session, HttpOnly, SameSite=Strict et Secure en HTTPS. Le frontend conserve seulement les informations du compte en mémoire et les restaure via GET /auth/session.

5.7.6. Fonctionnalité 6 : consulter les statistiques d'occupation

L'administrateur consulte les statistiques dans `/admin/analytics`. La page utilise `GET /analytics/overview` et `GET /analytics/workspaces` pour obtenir les totaux et l'état des espaces à partir des comptes, espaces, réservations et de l'heure courante.

Une réservation est active lorsque `startAt <= maintenant < endAt`. Le service compte les espaces distincts occupés et divise ce nombre par le total des espaces pour obtenir un taux entre 0 et 100 %.

L'API retourne HTTP 401 sans session, 403 pour un compte USER et 500 si le calcul échoue.

Tests associés : `analytics.service.spec.ts`, `admin.routes.spec.ts`, `authorized-api.spec.ts`.

La capacité sert à limiter les participants, mais elle n'entre pas dans le calcul du taux d'occupation : dès qu'une réservation est active, l'espace entier est compté comme occupé.

5.7.7. Fonctionnalité 7 : consulter les audits

La page `/admin/audit`, réservée aux administrateurs, affiche les suppressions enregistrées dans MongoDB, de la plus récente à la plus ancienne. Elle appelle `GET /audit?limit=100` ; la limite peut varier de 1 à 200. Chaque événement contient l'action, l'entité, les données supprimées, la date, ainsi que l'identifiant, l'adresse et le rôle de l'auteur.

Un utilisateur standard ne peut accéder ni à la page ni à cette route. L'API retourne HTTP 400 si la limite est invalide, 401 sans session, 403 pour un compte USER et 500 si la lecture échoue.

Tests associés : `audit.service.spec.ts`, `mongo-audit.integration.spec.ts`, `admin.routes.spec.ts`, `route-guard.spec.ts`.

5.7.8. Fonctionnalité 8 : inviter et gérer les participants

Depuis `/bookings`, le propriétaire ou un administrateur utilise les actions « Gérer » et « Inviter » pour ajouter un utilisateur déjà inscrit. La réservation ne doit pas être terminée et une place doit être disponible. L'invité peut accepter ou refuser. Une réservation publique propose aussi l'action « Rejoindre » aux utilisateurs connectés ; une réservation privée reste visible par ses membres.

Chaque participant possède un rôle OWNER ou GUEST, un statut PENDING, ACCEPTED ou DECLINED, ainsi que les dates de réponse et de check-in. Le service verrouille l'espace puis la réservation, contrôle la capacité et crée ou met à jour le participant. L'API retourne HTTP 403 pour une tentative de rejoindre une réservation privée, 404 si l'utilisateur n'existe pas et 409 en cas de doublon, de capacité atteinte ou de réservation terminée.

Tests associés : `booking-collaboration.routes.spec.ts`, `postgres-bookings.integration.spec.ts`, `authorized-api.spec.ts`, `booking-flow.spec.ts`.

6. SPÉCIFICATIONS TECHNIQUES

6.1. Référencement

L'accès à Tempo demande une connexion. Le HTML indique lang="fr" et contient <meta name="robots" content="noindex, nofollow"> pour demander aux moteurs de ne pas indexer les pages. Cette balise concerne le référencement ; la protection des données dépend des contrôles d'accès de l'application et de l'API.

6.2. Environnement technique

Domaine	Version	Usage
Runtime	Bun 1.3.14	TypeScript, scripts, paquets et workspaces
Backend	Hono 4.12.24	API HTTP, middlewares et RPC typé
Validation	Zod 4.4.3	Corps, paramètres et chaînes de requête
Frontend	Svelte 5.56.3, SvelteKit 2.63.1	Pages, composants et navigation
Build	Vite 7.3.5	Développement et production
Interface	Tailwind CSS 4.3.0, shadcn-svelte	Mise en page et composants
SQL	Drizzle ORM 0.45.2, Drizzle Kit 0.31.10	Schéma, requêtes et migrations
QR code	qrcode 1.5.4	Génération du QR sous forme de Data URL
Base relationnelle	PostgreSQL 18.6, Alpine 3.24	Comptes, espaces, réservations, participants et QR
Base documentaire	MongoDB 8.0.29 en configuration locale ; 7.0.43, Jammy sur le serveur	Audits de suppression ; adaptation au noyau de l'hôte
Qualité	Oxlint 1.68.0, Oxfmt 0.21.0	Lint et formatage
Tests	Bun Test, Vitest 4.1.8, Playwright 1.57.0	Tests isolés, intégrations et E2E
CI	GitHub Actions	Contrôles qualité et recette Docker
Publication	Docker Compose, Caddy 2.11.4 observé, DuckDNS	Reverse proxy, HTTPS et sous-domaine public

Je travaille sous Windows avec PowerShell. La CI fonctionne sur un runner Ubuntu fourni par GitHub.

Environnement	Composition
Développement	Backend et frontend lancés avec Bun, PostgreSQL et MongoDB locaux
Test	Mocks, bases PostgreSQL et MongoDB réelles, Chromium piloté par Playwright

Environnement	Composition
Démonstration	Quatre services Docker Compose et profil optionnel de seed
Démonstration publique	Serveur Ubuntu partagé, cinq services Compose, proxy Caddy existant et domaine DuckDNS

6.2.1. Ports et configuration

Ces ports correspondent à la configuration locale `docker-compose.yml`. Sur le serveur, les applications et les bases restent dans le réseau Docker.

Service	Port hôte	Port du conteneur
PostgreSQL	5432	5432
MongoDB	27017	27017
API Hono	3000	3000
Frontend SvelteKit	5173	3000

Les modèles de configuration se trouvent dans `.env.example` et `apps/backend/.env.example`. Les fichiers `.env` réels sont ignorés par Git.

Groupe	Variables
PostgreSQL	POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, DATABASE_URL
MongoDB	MONGO_INITDB_ROOT_USERNAME, MONGO_INITDB_ROOT_PASSWORD, MONGO_URL, MONGO_DB_NAME
Authentification et HTTP	JWT_SECRET, FRONTEND_ORIGIN, AUTH_RATE_LIMIT_MAX, AUTH_RATE_LIMIT_WINDOW_MS, TRUST_PROXY
Frontend	PUBLIC_API_URL
Démonstration	DEMO_ADMIN_EMAIL, DEMO_ADMIN_PASSWORD, DEMO_USER_EMAIL, DEMO_USER_PASSWORD

6.2.2. Déploiement public du 22 septembre 2026

La démonstration est hébergée sur une machine Ubuntu mise à disposition par un ami. Les fichiers du projet, les secrets et les données persistantes sont dans `/opt/tempo/`. Docker gère les images, les conteneurs et les journaux en dehors de ce dossier. Le proxy partagé conserve ses certificats dans son propre stockage.

Composant	Rôle et exposition sur le serveur
Caddy de l'hôte	Reçoit HTTP/HTTPS sur les ports existants 80 et 443 ; gère le certificat du domaine
Passerelle Caddy de Tempo	Écoute sur <code>127.0.0.1:18080</code> ; transmet <code>/api/*</code> au

Composant	Rôle et exposition sur le serveur
	backend en retirant /api, les autres chemins au frontend
Frontend SvelteKit et API Hono	Port 3000 interne à chacun de leurs conteneurs ; aucune publication directe sur l'hôte
PostgreSQL 18.6	Port 5432 interne au réseau Docker ; données sous /opt/tempo/data/postgres
MongoDB 7.0.43	Port 27017 interne au réseau Docker ; données sous /opt/tempo/data/mongo et /opt/tempo/data/mongo-config

J'ai créé le sous-domaine gratuit tempo-val.duckdns.org dans DuckDNS et renseigné l'IPv4 du serveur. Le proxy Caddy charge les fichiers /srv/*/caddy.conf. Le fichier /srv/tempo/caddy.conf lui indique de transmettre les requêtes de Tempo à son port local. La configuration a été validée puis rechargée sans remplacer celle des autres sites. Caddy gère HTTPS ; les échanges entre les composants passent par les réseaux internes.

Les secrets sont dans .env.host, avec des droits 600 qui en réservent la lecture à root.

TEMPO_ORIGIN=https://tempo-val.duckdns.org définit l'origine autorisée par l'API et l'adresse publique du frontend. Celui-ci appelle l'API sous /api. Les fichiers d'environnement ont été transférés séparément, après autorisation, et ne sont pas inclus dans les images. Pour limiter les tentatives de connexion, la passerelle transmet l'adresse du client en tenant compte du proxy privé.

MongoDB 8.0.29 refusait de démarrer à cause d'une incompatibilité avec le noyau Linux 7.0 du serveur, que les conteneurs utilisent aussi. Après vérification, j'ai fixé la version à 7.0.43-jammy dans deployment/compose.mongo7.yml. Comme la base était encore vide, il n'y avait aucune donnée à migrer. Les services ont ensuite démarré sans changement de noyau ni redémarrage de la machine.

La commande de démarrage à conserver inclut cette adaptation :

```
cd /opt/tempo
docker compose --env-file .env.host -f compose.host.yml -f deployment/compose.mongo7.yml up -d --no-build --wait
```

Le backend applique les migrations PostgreSQL au démarrage. Le seed, lancé ensuite par SSH dans le conteneur backend, a créé les comptes admin@tempo.local et user@tempo.local, les bureaux Horizon et Rivage et les salles Atlas et Boréale. Il ajoute aussi une réservation publique dans Atlas le 15 décembre 2027, de 13 h à 15 h UTC, avec son propriétaire accepté et un invité en attente. Ce jeu de données est indépendant de la base locale. Relancer le seed réinitialise les mots de passe et les rôles des comptes de démonstration.

J'ai gardé les anciennes configurations dans deployment/before-https/, sans les données des bases. Pour retirer Tempo du serveur partagé, il faut d'abord supprimer /srv/tempo/caddy.conf, vérifier la configuration de Caddy et la recharger. On peut ensuite arrêter le projet Compose et supprimer son dossier. Il faut éviter un nettoyage Docker global, qui toucherait aussi les autres projets. Les sauvegardes automatiques et leur copie hors du serveur restent à mettre en place.

6.3. Navigation et accessibilité

En local, docker compose up --build --detach --wait lance les services. Le frontend est accessible sur <http://localhost:5173> et l'API sur <http://localhost:3000>. La démonstration publiée le 22 septembre 2026 est accessible sur <https://tempo-val.duckdns.org>, avec l'API sous /api.

Un utilisateur doit être connecté pour ouvrir /bookings ou /check-in, et avoir le rôle ADMIN pour les pages /admin/*. Le frontend gère ces redirections. L'API vérifie les mêmes droits à chaque appel.

6.3.1. Routes de l'API

Méthode	Route	Droit	Fonction
GET	/health	Public	Vérifier que l'API répond
POST	/auth/register	Public, limité par adresse	Créer un compte USER
POST	/auth/login	Public, limité par adresse	Créer le cookie de session et obtenir le compte
GET	/auth/session	Public	Restaurer le compte ou retourner user: null
POST	/auth/logout	Public, protection CSRF	Supprimer le cookie de session
GET	/users	ADMIN	Lister les comptes
POST	/users	ADMIN	Créer un compte sans renvoyer son hash
GET	/workspaces	Connecté	Lister les espaces
GET	/workspaces/:id	Connecté	Consulter un espace
POST	/workspaces	ADMIN	Créer un espace
PATCH	/workspaces/:id	ADMIN	Modifier un espace
DELETE	/workspaces/:id	ADMIN	Supprimer et tenter d'auditer
GET	/bookings	Connecté	Lister les réservations visibles et les invitations
POST	/bookings	Connecté	Créer une réservation publique ou privée
POST	/bookings/:id/invitations	Propriétaire ou ADMIN	Inviter un utilisateur
PATCH	/bookings/:id/invitations/:participantId	Utilisateur invité	Accepter ou refuser
POST	/bookings/:id/join	Connecté	Rejoindre une réservation publique

Méthode	Route	Droit	Fonction
POST	/bookings/:id/qr	Propriétaire ou ADMIN	Générer ou renouveler le QR
POST	/bookings/:id/check-in	Participant accepté	Enregistrer la présence
DELETE	/bookings/:id	Propriétaire ou ADMIN	Supprimer et tenter d'auditer
GET	/analytics/overview	ADMIN	Obtenir les totaux et le taux actuel
GET	/analytics/workspaces	ADMIN	Obtenir l'état de chaque espace
GET	/audit?limit=100	ADMIN	Consulter les audits

6.4. Services tiers

Aucun service tiers métier n'est appelé. Tempo n'envoie pas d'email et n'utilise ni CRM, ni analytics externe, ni réseau social. GitHub Actions exécute uniquement la CI. Pour l'hébergement, DuckDNS fournit le sous-domaine et sa résolution DNS. Caddy automatise la gestion du certificat HTTPS. Le déploiement reste manuel.

6.5. Sécurité

Les rôles ADMIN et USER sont enregistrés dans PostgreSQL et repris dans le JWT. authGuard vérifie ce jeton ; adminGuard renvoie HTTP 403 si un compte USER tente d'appeler une route d'administration. Les services vérifient les droits liés à une action précise, par exemple si le compte possède la réservation ou s'il est bien le destinataire de l'invitation.

Les JWT sont signés en HS256 avec hono/jwt et expirent après 24 heures. JWT_SECRET est obligatoire au démarrage. Les mots de passe sont hachés avec Bun.password.hash et vérifiés avec Bun.password.verify. Les réponses de l'API ne contiennent jamais le hash.

Le CORS accepte uniquement FRONTEND_ORIGIN. Hono ajoute une politique CSP et les en-têtes X-Content-Type-Options, Referrer-Policy, X-Frame-Options, HSTS et Permissions-Policy aux réponses de l'API. Les en-têtes des pages SvelteKit doivent être vérifiés séparément. Par défaut, l'inscription et la connexion sont limitées à 10 requêtes par adresse en 15 minutes. Au-delà, l'API retourne HTTP 429 avec Retry-After.

Les données reçues passent d'abord par Zod. La base applique ensuite ses propres règles : clés étrangères, contraintes CHECK, un seul participant par utilisateur et réservation, et aucun chevauchement entre réservations concurrentes.

Le jeton QR contient 256 bits aléatoires. L'URL le place dans le fragment, puis la page le retire de l'historique après un check-in réussi. Si une connexion est nécessaire, la destination passe par sessionStorage et non par la chaîne de requête de /login. PostgreSQL ne conserve que le hash SHA-256. Le backend vérifie le participant, son statut, le créneau et l'expiration du jeton.

6.5.1. Limites connues

- Le cookie de session est HttpOnly, SameSite=Strict et Secure lorsque FRONTEND_ORIGIN utilise HTTPS. Les mutations exigent une origine autorisée et un en-tête X-CSRF-Protection: 1. Le frontend et l'API doivent être déployés sur le même site au sens du navigateur. Une faille XSS pourrait encore effectuer des actions avec la session : le cookie ne remplace pas la prévention XSS.
- Il n'existe pas de révocation individuelle des JWT ni de rotation automatique du secret.
- Le limiteur est en mémoire. Plusieurs instances backend devraient partager son état.
- Le développement local utilise HTTP. Depuis le 22 septembre, le déploiement public termine TLS sur le Caddy de l'hôte et redirige HTTP vers HTTPS. Les échanges internes restent en HTTP.
- Les sauvegardes sont documentées mais ne sont ni planifiées ni externalisées.
- L'audit MongoDB est best effort. Une panne ne revient pas sur une suppression PostgreSQL déjà validée.
- Le QR est commun aux participants d'une réservation. Un utilisateur doit tout de même être connecté, accepté et dans le créneau, mais un participant peut transmettre le code ou effectuer le check-in à distance.

GitHub Actions vérifie le format, le lint, les types, les tests, les builds et Docker Compose. J'ai choisi de laisser la branche main sans protection pendant le développement.

7. RÉALISATIONS

J'ai sélectionné les extraits de code suivants pour expliquer les principaux traitements. Les extraits 7.2 et 7.3 correspondent à la version antérieure aux corrections de session et de capacité. Les explications sous le code précisent les changements apportés depuis.

7.1. Détection des chevauchements

7.1.1. Affichage

Toutes les réservations

Supervisez les réservations de l'ensemble des utilisateurs

← Accueil  Espaces  Audit admin@tempo.local →

 Nouvelle Réservation

 **Erreur**
Ce créneau est déjà réservé pour cet espace

 RECETT... 

Privée 

24-Sep-2026 

10:00 

24-Sep-2026 

11:00 

Réserver

2 réservations

Espace	Propriétaire	Début	Fin	Visibilité	Participants	Statut	Actions
 Salle Atlas Salle · 6 pers.	admin@tempo.local ADMIN	15/12/2027 14:00	15/12/2027 16:00	Publique	 2/6	À venir	 Gérer X Annuler
 RECETTE-2026-09-24 Bureau · 4 pers.	admin@tempo.local ADMIN	24/09/2026 09:00	24/09/2026 18:00	Privée	 1/4	En cours	 Gérer X Annuler

Refus d'un créneau déjà occupé sur la démonstration publique, le 24 septembre 2026.

7.1.2. Extrait de code

```
async checkOverlap(
  workspaceId: number,
  startAt: Date,
  endAt: Date,
  excludeBookingId?: string,
): Promise<boolean> {
  const conditions = [
    eq(bookings.workspaceId, workspaceId),
    lt(bookings.startAt, endAt),
    gt(bookings.endAt, startAt),
  ];

  const overlapping = await db.query.bookings.findFirst({
    where: excludeBookingId
      ? and(...conditions, ne(bookings.id, excludeBookingId))
```

```

        : and(...conditions),
    });

    return !!overlapping;
}

```

7.1.3. Argumentation

Deux réservations se chevauchent si la nouvelle commence avant la fin de l'autre et se termine après son début. La règle détecte aussi bien deux créneaux identiques qu'un créneau inclus dans un autre ou un recouvrement partiel. En revanche, une réservation peut commencer au moment où la précédente se termine.

Le contrôle dans le service permet de refuser rapidement un créneau déjà pris. Mais deux requêtes peuvent passer ce contrôle au même moment. C'est alors la contrainte PostgreSQL `bookings_workspace_time_exclusion` qui empêche le doublon. Le backend transforme l'erreur SQL 23P01 en `BOOKING_OVERLAP` et renvoie HTTP 409.

7.2. Authentification et contrôle des rôles

7.2.1. Affichage

/

7.2.2. Extrait de code

```

export const authGuard = jwt({
  secret: authService.getSecret(),
  alg: 'HS256',
});

export const adminGuard: MiddlewareHandler<AuthEnv> = async (c, next) => {
  const payload = c.get('jwtPayload');

  if (payload.role !== 'ADMIN') {
    return c.json({ error: 'Admin access required' }, 403);
  }

  await next();
};

```

7.2.3. Argumentation

Cet extrait utilise l'ancien middleware `jwt`. Aujourd'hui, `authGuard` appelle `readSession` pour lire le cookie `tempo_session` et vérifier la signature HS256 ainsi que le contenu du jeton. Le résultat est placé dans `jwtPayload`. Le garde CSRF de `app.ts` contrôle les requêtes de modification. `adminGuard` vérifie le rôle administrateur ; les services utilisent ensuite l'identifiant `sub` pour les droits propres à une réservation ou à une invitation.

L'authentification permet de savoir qui est connecté. Les autorisations déterminent ensuite les actions auxquelles ce compte a droit. Les tests vérifient plusieurs routes protégées avec un compte USER, un compte ADMIN et sans jeton.

7.3. Capacité et concurrence des participants

7.3.1. Affichage

Participants et QR code
RECETTE-2026-09-24 · 24/09/2026 09:00

Participants

admin@tempo.local
Organisateur

Accepté

user@tempo.local
Invité

En attente

participant@entreprise.fr

Inviter

Générer le QR code

Invitation en attente dans le panneau des participants, le 24 septembre 2026.

7.3.2. Extrait de code

```
return await db.transaction(async (transaction) => {
  await transaction.execute(
    sql`SELECT "id" FROM "bookings" WHERE "id" = ${bookingId} FOR UPDATE`,
  );

  const booking = await transaction.query.bookings.findFirst({
    where: eq(bookings.id, bookingId),
    with: { workspace: true },
  });

  if (!booking) throw new Error('BOOKING_NOT_FOUND');

  const reservedPlaces = await countReservedPlaces(transaction, bookingId);
  if (reservedPlaces >= booking.workspace.capacity) {
    throw new Error('BOOKING_FULL');
  }

  // insertion ou réactivation du participant
});
```

7.3.3. Argumentation

Une invitation en attente réserve déjà une place. Si deux demandes arrivent pour la dernière place libre, elles pourraient toutes les deux être acceptées sans verrou. `lockAdmission` verrouille donc l'espace puis la réservation, recompte les participants qui n'ont pas refusé et enregistre l'ajout. L'extrait ci-dessus montre seulement le verrou de la réservation. Le code utilise aussi celui de l'espace pour tenir compte d'une modification de capacité au même moment.

Les invitations et le bouton permettant de rejoindre une réservation publique passent par ce même contrôle. Le propriétaire, lui, est ajouté avec le statut accepté dans la transaction qui crée la réservation.

7.4. Jeton de check-in et stockage du hash

7.4.1. Affichage

Participants et QR code
RECETTE-2026-09-24 · 24/09/2026 09:00

Participants

admin@tempo.local
Organisateur

Accepté

user@tempo.local
Invité

En attente

participant@entreprise.fr

Inviter

Renouveler le QR code



Chaque participant accepté peut scanner ce code pendant le créneau.

QR code de la réservation de recette, le 24 septembre 2026. La réservation a été supprimée après le contrôle.

Check-in Tempo



Présence confirmée

Check-in enregistré le 24/09/2026 10:35:58.

Retour aux réservations

Confirmation du check-in du propriétaire pendant le créneau, le 24 septembre 2026.

7.4.2. Extrait de code

```
function hashToken(token: string): string {
    return new Bun.CryptoHasher('sha256').update(token).digest('hex');
}

function createToken(): string {
    const bytes = crypto.getRandomValues(new Uint8Array(32));
    return Array.from(bytes, (byte) => byte.toString(16).padStart(2, '0')).join('');
}

if (!participant) throw new Error('PARTICIPANT_NOT_FOUND');
if (participant.invitationStatus !== 'ACCEPTED') {
    throw new Error('INVITATION_NOT_ACCEPTED');
}

const now = new Date();
if (now < participant.booking.startAt) throw new Error('CHECK_IN_TOO_EARLY');
if (now >= participant.booking.endAt) throw new Error('BOOKING_ENDED');
```

7.4.3. Argumentation

Le QR contient le jeton complet, alors que la base n'en garde que le hash. Quand le serveur reçoit le jeton, il calcule son hash et le compare à celui de la base. Quelqu'un qui récupère cette table ne dispose donc pas directement d'un jeton utilisable. Lorsqu'un nouveau QR est généré, son hash remplace l'ancien et le précédent code ne fonctionne plus.

Un QR partagé ne prouve pas la présence dans la salle. L'utilisateur doit être connecté, appartenir à la réservation, avoir accepté l'invitation et effectuer l'action pendant le créneau.

7.5. Audits MongoDB

7.5.1. Affichage

La page /admin/audit montre les 100 événements les plus récents avec l'action, l'entité, la date et l'auteur.

Journal d'audit

← Accueil 📅 Réservations 🗑 Espaces 📊 Analytique admin@tempo.local →

Les 100 suppressions sensibles les plus récentes

2 événements

Les événements enregistrés dans MongoDB conservent l'identité de leur auteur.

Date	Action	Élément	Auteur	Rôle
24/09/2026 15:38:52	Espace supprimé	RECETTE-2026-09-24 ID : 5	admin@tempo.local	ADMIN
24/09/2026 15:38:52	Réservation annulée	Espace #5 ID : af5bb8f2-3496-466e-831b-444c1b4590a5	admin@tempo.local	ADMIN

Journal d'audit après suppression par l'API de la réservation et de l'espace de recette, le 24 septembre 2026.

7.5.2. Extrait de code

```
async logDeletion(
    entityType: AuditLog['entityType'],
    entityId: string | number,
    entityData: Record<string, unknown>,
    performedBy: AuditLog['performedBy'],
): Promise<void> {
```

```

const actionMap: Record<AuditLog['entityType'], AuditAction> = {
  workspace: 'DELETE_WORKSPACE',
  booking: 'DELETE_BOOKING',
  user: 'DELETE_USER',
};

await this.log({
  action: actionMap[entityType],
  entityType,
  entityId,
  entityData,
  performedBy,
});
}

```

7.5.3. Argumentation

J'ai réservé PostgreSQL aux données métier et MongoDB aux audits. Chaque audit conserve les données de l'élément supprimé, qui varient selon son type, ainsi que l'auteur et la date de suppression. Le journal affiche les événements du plus récent au plus ancien.

Si MongoDB est indisponible, la suppression dans PostgreSQL a quand même lieu. Il peut donc manquer une entrée dans le journal d'audit. Cette limite est décrite dans la documentation et couverte par les tests.

7.6. Client RPC typé

7.6.1. Affichage

/

7.6.2. Extrait de code

```

import { env } from '$env/dynamic/public';
import { hc } from 'hono/client';
import type { AppType } from '@tempo/backend/src/index';

export function createApiClient(apiUrl: string | undefined, options: ApiClientOptions = {}) {
  return hc<AppType>(normalizeApiUrl(apiUrl), {
    init: { credentials: 'include' },
    headers: { 'X-CSRF-Protection': '1' },
    fetch: options.fetch,
  });
}

```

7.6.3. Argumentation

Le client frontend importe AppType depuis le backend. Hono en déduit les types des routes, des paramètres et des réponses. Quand je change une route de façon incompatible, TypeScript m'indique quels appels du frontend doivent être corrigés. Je n'ai pas besoin de générer un SDK pour partager ces types.

Le frontend et le backend étant dans le même dépôt, ils peuvent partager ces types. Cela ne dispense pas de vérifier les données à l'exécution avec Zod et de contrôler les réponses HTTP à leur réception.

8. ÉLÉMENTS DE SÉCURITÉ DE L'APPLICATION

La sécurité repose sur plusieurs contrôles complémentaires :

Risque	Mesure appliquée	Preuve
Mot de passe exposé	Hash avec Bun.password, aucun hash renvoyé	auth.service.spec.ts, users.service.spec.ts
Route appelée sans session	JWT HS256 avec expiration de 24 heures	auth.guard.spec.ts, tests de routes
Action d'administration par USER	adminGuard et contrôles de rôle	admin.routes.spec.ts
Donnée malformée	Schémas Zod sur corps, paramètres et requêtes	Tests DTO et HTTP 400
Double réservation concurrente	Exclusion GIST PostgreSQL	Test d'intégration concurrent
Dépassement de capacité	Transaction et verrou FOR UPDATE	Service des participants
Bruteforce sur l'authentification	10 requêtes par adresse sur 15 minutes	rate-limit.spec.ts
Appel depuis une origine imprévue	CORS limité à FRONTEND_ORIGIN	app.security.spec.ts
Jeton QR lu en base	Hash SHA-256 et rotation	Service de check-in
Jeton QR transmis au serveur frontend	Fragment URL puis sessionStorage si connexion	route-guard.spec.ts, E2E
Secret versionné	Variables d'environnement et modèles factices	.gitignore, .env.example
Régression	Format, lint, types, tests, builds et recette Docker en CI	GitHub Actions

Le JWT est conservé dans un cookie `HttpOnly` de 24 heures, inaccessible au JavaScript de la page. Le client RPC utilise `credentials: include` et un en-tête dédié à la protection CSRF. L'API vérifie aussi l'origine exacte des requêtes qui modifient les données. La déconnexion supprime le cookie ; elle ne révoque pas un JWT qui aurait été copié avant sa suppression.

Le limiteur de requêtes garde son état dans le processus backend. Si je lançais plusieurs instances, il faudrait qu'elles partagent cet état. Pour la démonstration publique, le Caddy de l'hôte termine la connexion TLS avant l'application. La V1 garde aussi deux limites : un audit peut manquer si MongoDB est indisponible, et les participants utilisent le même QR pour une réservation.

9. PLAN DE TESTS

9.1. Niveaux de test

Niveau	Outil	Périmètre	Nombre
Backend unitaire et HTTP	Bun Test	Services, DTO, middlewares, routes, sécurité	106
Frontend unitaire	Vitest	Authentification, client RPC, API autorisée, gardes	22
Intégration PostgreSQL	Bun Test	Persistance, concurrence, collaboration et QR	15
Intégration MongoDB	Bun Test	Écriture, auteur, ordre et filtrage	2
E2E Chromium	Playwright	Réservation et parcours collaboratif	2

Le backend compte 123 tests : 106 tests unitaires et HTTP, 15 tests PostgreSQL et 2 tests MongoDB. Les suites d'intégration se lancent séparément.

9.2. Couverture par module

Module	Fichiers principaux	Points contrôlés
Authentification	auth.service.spec.ts, demo-seed.config.spec.ts	Inscription, connexion, hash, JWT, configuration
Sécurité HTTP	app.security.spec.ts, security.config.spec.ts, rate-limit.spec.ts	CORS, en-têtes, origine, limite et HTTP 429
Autorisations	auth.guard.spec.ts, admin.routes.spec.ts	HTTP 401, HTTP 403, rôles USER et ADMIN
Espaces	workspaces.service.spec.ts, workspaces.dto.spec.ts	CRUD, capacité et validation PATCH
Réservations	bookings.service.spec.ts, http.routes.spec.ts	Création, visibilité, chevauchement, suppression et statuts HTTP
Collaboration	booking-collaboration.routes.spec.ts	Invitation, réponse, participation publique, droits et fenêtre de check-in
Statistiques	analytics.service.spec.ts	Totaux, bornes temporelles et taux
Audit	audit.service.spec.ts	Écriture et lecture MongoDB
Frontend	quatre fichiers .spec.ts	Restauration de session, déconnexion, erreurs, client et navigation
Intégration	fichiers du dossier integration	Bases réelles et migrations
E2E	e2e/booking-flow.spec.ts	Deux parcours utilisateur complets

9.3. Environnements et critères de réussite

Les tests unitaires utilisent des mocks et n'ont pas besoin de bases de données. Les tests d'intégration tournent avec PostgreSQL et MongoDB. Pour les parcours navigateur, Playwright démarre les deux applications et pilote Chromium. Le job Docker reconstruit un environnement neuf, applique les migrations et charge le seed avant ses contrôles.

Pour valider une version, tous les tests doivent passer, Svelte Check ne doit signaler ni erreur ni avertissement, et le build de production doit réussir. Les parcours E2E contrôlent les réponses HTTP et ce qui s'affiche à l'écran. Si un parcours échoue, Playwright conserve une trace, une capture et une vidéo pour comprendre ce qui s'est passé.

Le 17 septembre 2026, 106 tests backend, 22 tests frontend, 15 tests PostgreSQL et 2 parcours Chromium ont réussi en local, avec les types, le lint, le formatage et le build. MongoDB était indisponible dans cet environnement isolé : sa persistance n'a pas été validée par cette exécution. Le 23 septembre, les deux jobs de la CI n° 78 ont réussi sur le commit 8d0d1cd. La capture et le lien de cette exécution figurent en section 1.3.3.

9.3.1. Vérifications du déploiement du 22 septembre 2026

Avant la publication, le formatage, le lint, les 106 tests backend et les 22 tests frontend ont réussi en local. Les deux images applicatives ont ensuite été construites sur le serveur. Le tableau ci-dessous reprend les vérifications faites sur la démonstration ; les tests d'intégration et Playwright n'ont pas été relancés à cette occasion.

Vérification	Résultat observé
Services Docker	PostgreSQL, MongoDB 7, backend et frontend déclarés sains ; passerelle démarrée
Site public	HTTP 200 sur https://tempo-val.duckdns.org/ , avec vérification du certificat TLS
Redirection	HTTP 308 vers HTTPS
API	/api/health renvoie OK
Accès sans session	/api/auth/session renvoie user: null ; la route protégée /api/bookings renvoie 401 lors du contrôle privé préalable
Cookie	Attributs HttpOnly, Secure et SameSite=Strict observés sur HTTPS
Seed	2 comptes, 4 espaces, 1 réservation et 2 participations dans PostgreSQL
Authentification	Connexion puis restauration de session vérifiées via HTTPS pour les rôles ADMIN et USER

Le contrôle du 24 septembre 2026 sur le site public a confirmé la création d'une réservation privée, le refus d'un chevauchement, l'envoi et l'acceptation d'une invitation, puis le check-in. La route /api/users a renvoyé 401 sans session et 403 avec le compte USER. La réservation et l'espace RECETTE-2026-09-24

ont été supprimés par l'API et les deux suppressions apparaissent dans le journal d'audit. L'annulation depuis le navigateur est restée bloquée au niveau de la boîte de confirmation. Il reste à tester la restauration d'une sauvegarde et la charge.

9.4. Évolutions du plan

Les tests actuels vérifient les règles fonctionnelles et les contrôles de sécurité de la V1. Avant d'aller au-delà de la démonstration, il reste à tester la charge, l'accessibilité et les parcours sous Firefox et WebKit.

10. JEU D'ESSAI DE LA FONCTIONNALITÉ LA PLUS REPRÉSENTATIVE

10.1. Fonctionnalité retenue

J'ai choisi de détailler une réservation publique avec invitation, acceptation et check-in par QR code. Pour terminer ce parcours, l'utilisateur doit se connecter, avoir les bons droits et respecter la capacité de l'espace. Il permet de vérifier ensemble les transactions PostgreSQL, la visibilité de la réservation et les écrans Svelte. D'autres tests du même module portent sur les chevauchements.

10.2. Scénarios

Référence	Action	Résultat attendu
JE1	Créer une réservation sur un espace existant et un créneau libre	HTTP 201, propriétaire ajouté comme participant accepté
JE2	Créer la même réservation avec deux requêtes simultanées	Une seule création réussit, l'autre reçoit HTTP 409
JE3	Créer une réservation sur un espace inexistant	HTTP 404 WORKSPACE_NOT_FOUND
JE4	Créer un créneau identique ou partiellement chevauchant	HTTP 409 BOOKING_OVERLAP
JE5	Créer un créneau qui commence à la fin du précédent	HTTP 201, les intervalles sont consécutifs
JE6	Inviter un utilisateur existant lorsqu'une place est libre	HTTP 201, participant PENDING
JE7	Rejoindre une réservation privée sans invitation	HTTP 403
JE8	Accepter l'invitation avec le compte concerné	HTTP 200, statut ACCEPTED
JE9	Générer le QR avec le propriétaire ou un administrateur	HTTP 200, QR généré et jeton utilisable pour la réservation
JE10	Effectuer le check-in avant le début du créneau	HTTP 409
JE11	Effectuer le check-in pendant le créneau avec le bon participant	HTTP 200 et checkedInAt enregistré
JE12	Annuler avec un autre utilisateur standard	HTTP 403
JE13	Annuler avec le propriétaire ou un administrateur	HTTP 200 et tentative d'audit

10.3. Résultats

Groupe	Scénarios validés	Résultat
bookings.service.spec.ts	JE1, JE3, JE4, JE5, JE12, JE13	Conforme
booking-collaboration.routes.spec.ts	JE6, JE7, JE8, JE10	Conforme
Intégration PostgreSQL	JE1, JE2, JE6, JE8, JE9, JE11	Conforme
E2E réservation	JE1 et JE13 depuis l'interface	Conforme
E2E collaboration	JE6, JE8, JE9 et JE11 depuis l'API et l'interface	Conforme
Intégration MongoDB	Audit de JE13	Résultat historique, suite non rejouée le 17 septembre

Les tests PostgreSQL appliquent les migrations, exécutent le parcours et vérifient checkedInAt en base. Le test E2E utilise les comptes du seed, accepte l'invitation dans l'interface, ouvre le lien du QR et attend le message « Présence confirmée ».

Tempo CI Pipeline

docs: improve mobile responsiveness with reduced padding and vertical... #78

Summary

All jobs

Triggered via push yesterday

Vaalley pushed · 8d0d1cd · main

Status: Success

Total duration: 2m 45s

Artifacts: -

ci.yml

on: push

Quality & Tests 40s

Docker Build Check 1m 57s

Quality & Tests summary

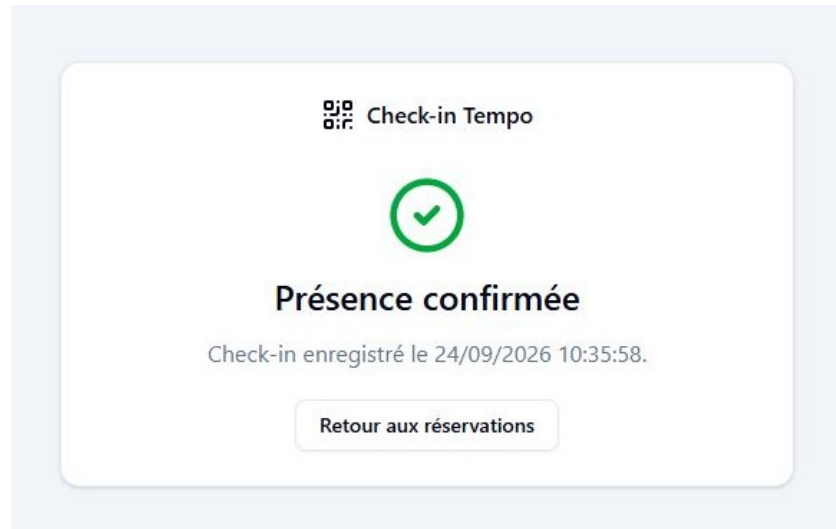
Vitest Test Report

Summary

- Test Files: 4 passes · 4 total
- Test Results: 22 passes · 22 total

Job summary generated at run-time

GitHub Actions n° 78, commit 8d0d1cd, 23 septembre 2026 : les deux jobs ont réussi.



Check-in confirmé sur la démonstration publique, le 24 septembre 2026. Le parcours manuel illustré utilise une réservation privée avec invitation.

10.4. Conclusion

Les scénarios automatisés lancés en local le 17 septembre ont réussi. Lors de la recette du 21 septembre, le refus des chevauchements, le contrôle de capacité et le check-in après acceptation de l'invitation ont aussi été vérifiés. Pour autoriser le check-in, le serveur contrôle le compte, sa participation, la réservation, le créneau et le jeton QR.

Le contrôle mobile du 24 septembre porte sur les écrans de la section 2.6.2. Pour la CI, le commit 8d0d1cd a passé l'exécution n° 78, mais le suivant est en échec : la version remise doit encore avoir sa propre exécution réussie. Il reste aussi à tester la charge. Les autres limites sont celles décrites plus haut : pas de révocation individuelle des JWT, un QR qui peut être partagé et des audits qui peuvent manquer si leur écriture échoue.

11. VEILLE SUR LES VULNÉRABILITÉS DE SÉCURITÉ

- *Discussions autour de la sécurité avec les collègues en entreprise*
- *Application du principe du moindre privilège pour les rôles applicatifs*
- *Choix de fonctions de hachage de mot de passe recommandées (Argon2id via Bun.password) plutôt que des algorithmes obsolètes*
- *Suivi des bonnes pratiques (validation des entrées, gestion des erreurs sans fuite d'information, expiration des jetons JWT)*